

IoT et Consommation

Objets connectés à faible consommation

Pierre COURBIN
pierre.courbin@esiea.fr

esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Références et contexte

Références et sources

Si vous repérez une erreur dans les affirmations suivantes, n'hésitez pas à me l'indiquer en commentaire de ce Deck !

- Sur chaque image, vous trouverez le lien vers sa source
- Suite à chaque citation, vous trouverez le lien vers sa source
- De façon générale, je n'ai aucun lien avec les différentes sources listées, ce sera sinon indiqué clairement.
  J'ai parfois simplement trouvé leur image particulièrement pertinente...

Conseils de ressources

- Advanced ESP32, By Venkatesh Varadachari, Naveen PS, February 2020 (Cours en vidéo, total de 3h33)

À propos de ce cours

Ce cours est donné aux étudiants de Master2 (3ème année d'école d'ingénieur) qui ont choisi la majeure "Systèmes Embarqués".
Les étudiants y étudient l'aspect "Architecture", "Certification" des systèmes embarqués, mais aussi l'IA, la Sécurité, la Sureté et le Co-Design Hardware/Software.

À propos de l'enseignant

Informations professionnelle sur LinkedIn et sur l'aspect recherche sur ResearchGate.

À propos de l'ESIEA

L'ESIEA est une école d'ingénieurs CTI dédiée à l'enseignement et à la recherche en sciences et technologies du numérique.
Elle recrute principalement au niveau postbac et au niveau post bac+2 et forme des ingénieurs opérationnels s'insérant parfaitement dans le monde professionnel.
Elle est implantée à Paris, Ivry- sur-Seine et Laval et offre une formation d'ingénieurs dans **5 grandes thématiques** : Cybersécurité, Data et IA, Systèmes embarqués, Ingénierie logicielle et Réalité virtuelle.
L'école propose aussi un Bachelor, un Mastère Spécialisé et des formations continues en Cybersécurité ainsi qu'un MSc en Intelligence Artificielle.
L'ESIEA est labélisée EESPIG et est membre de la CGE, de l'UGEI, de la CDEFI et de Campus France.



esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Plan du cours

On va faire quoi en fait ?

1. Introduction Générale

- **Déroulé du cours**
- **(ré)Introduction à l'IoT**

2. Objet : plateformes et architectures

- **Plateformes et Architectures**
- **Cas du ESP32**

3. Connectivité : réseaux et services Cloud

- Communication par bus et HAN/LAN/WAN
- Principe d'API
- Services Cloud

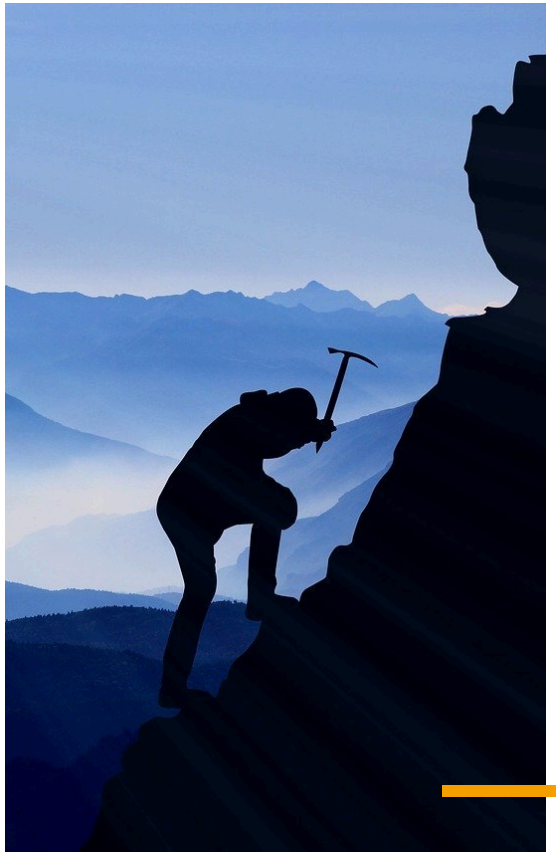
4. Objet : consommation

- ESP32 et modes de consommations
- Principes de mesure de consommation

5. PoC : à vous de jouer

Déroulé du cours

3 séances pour découvrir, 3 séances pour réaliser



La question de la consommation d'un objet connecté est large :

- **Conception matérielle de l'objet**, choix de composants, etc. (Peu évoqué dans ce cours)
- **Conception logicielle de l'objet**, choix des modes de consommations, etc. (Focus lors des cours, TPs et PoC)
- **Conception de l'architecture globale** : nombre d'objets pour répartir les fonctions, protocoles de communication, appels à des APIs, spécifications temporelles des fonctionnalités (Focus lors des cours et lors du PoC)

	Cours	TP	PoC	Évaluation
Séance 1	(ré)Introduction IoT	(re)Découverte ESP32	-	-
Séance 2	Connectivité : réseaux et services Cloud	ESP32 et réseau	-	Quiz 1
Séance 3	Objet : plateformes et architectures	ESP32 et consommation	-	Quiz 2 Fiche relevés, Dossier de TP, auto-test
Séance 4	L'importance des spécifications	L'importance des spécifications	Définition du sujet et de l'équipe, Premières réalisations	Quiz 3
Séance 5	-	-	Avancement, Finalisation	
Séance 6	-	-	Soutenances et évaluations	

Déroulé du cours

Travaux Pratiques (TP), quelques détails...

Durant les séances 1 à 3, vous aurez des TPs à réaliser.

Ces TPs doivent vous permettre de mieux comprendre l'intérêt, la mise en œuvre et la complexité de chaque partie du cours.
Ils devraient aussi vous donner les briques essentielles vous permettant de mener à bien votre PoC.

Le TP n'est pas un devoir à rendre, c'est l'occasion de comprendre.
Ce que vous aurez compris pendant la séance est ce qui vous rapportera des points. Ce que vous aurez généré ne vous en rapportera aucun.

- **Vous rendez un dossier, pas un rapport rédigé.**
  Votre fiche de relevés, vos preuves (captures, photos, schémas, justification de vos broches), et 2 pages d'analyse sur un sujet imposé.
  Le code va dans le dépôt Git.
  8 pages maximum.

- **Le dossier n'est pas noté sur l'exactitude de vos réponses.**
  Il est noté sur la traçabilité de vos preuves et sur la qualité de votre raisonnement dans la note d'analyse.
  La grille est sur Moodle dès aujourd'hui, c'est la même que pour le rapport de PoC.

- **Votre compréhension est évaluée individuellement, en salle.**
  Un quiz en ouverture des séances 2 et 3, un contrôle en ouverture de la séance 4.
  Sur Moodle, internet bloqué, une feuille A4 manuscrite autorisée.

- **Chaque binôme a ses propres paramètres, déduits de son numéro.**
  Vos mesures sont les vôtres, à commencer par la valeur réelle de votre résistance.
  Plusieurs questions du contrôle porteront sur vos propres chiffres.

Déroulé du cours

Proof of Concept (PoC)

Pour montrer votre compréhension des concepts vus en cours (et surtout pour mieux les appréhender), vous allez réaliser un "PoC".

Quelques contraintes

- 2 à 4 étudiants par équipe
- 1 fonctionnalité par étudiant (au moins)
- Le système doit être **connecté à son environnement direct** (capteurs, LEDs, moteur, écrans...)
- Le système doit être **connecté à l'extérieur** (API, serveur web, un autre objet, un email,...)
- L'architecture globale et le système doivent être pensés et codés pour **optimiser la consommation énergétique**

Fonctionnalité ?

Par exemple, une équipe de 4 étudiants travaillant sur un "poulailler intelligent connecté" pourrait mettre en œuvre :

1. Un système de capteur infrarouge (ou pression) qui compte les poules (environnement direct)
2. Une page Web sécurisée qui affiche le flux vidéo à l'intérieur du poulailler et qui permet d'ouvrir/fermer manuellement la porte (extérieur)
3. Un système de capteur de lumière qui ouvre la porte le matin et qui la ferme le soir (environnement direct)
4. Un système de notification pour alerter lorsque de la nourriture ou de l'eau manque (monde numérique)

Le tout avec éventuellement différents objets, des fréquences de fonctionnement et des réflexions sur l'usage d'interruptions pour maximiser les temps de sommeil de chaque objet tout en garantissant une certaine réactivité.

Rendus

- Rapport (10/15 pages)
- Présentation
- Dépôt Git avec codes et tutoriels pour reproduire l'exemple

Déroulé du cours

Évaluation

Element	Sujet	Evaluateur	Pourcentage
Quiz et contrôle individuels	TPs	Enseignant	40
Dossier de preuves et note d'analyse	TPs	Enseignant	20
Rapport du PoC	PoC	Enseignant	15
Présentation	PoC	Enseignant	15
Présentation	PoC	Pairs	7.5
Evaluation des autres PoC	PoC	Automatique	2.5



Sujet	Pourcentage
TPs	60
PoC	40

Evaluateur	Pourcentage
Pairs	7.5
Automatique	2.5
Enseignant	90

Naturellement, la fraude, le plagiat, l'absence de sources, la malhonnêteté sont sanctionnés par un 0 à l'exercice et un conseil de discipline.

Utilisation d'un outil de type LLM est autorisé, pour un usage d'outils d'approfondissement. Et vos usages doivent être documenté dans votre rapport pour ne pas être considérés comme du plagiat.

Je vous recommande l'usage de ces logos.

Objet connecté ?

> *“ Un « objet » peut être **une personne** équipée d'un pacemaker, **un animal** de ferme qui porte une puce (transpondeur), **une voiture** qui embarque des capteurs pour alerter le conducteur lorsque la pression des pneumatiques est trop faible, ou encore **tout objet naturel ou fabriqué** par l'être humain auquel peuvent être attribuées **une adresse IP et la capacité de transférer des données sur un réseau**.*

Objet connecté, LeMagIT

L'avènement de l'IoT et conséquences

Avènement... des objets pensés comme connectés ?

“ Le terme Internet des objets est utilisé pour la première fois en 1999 par Kevin Ashton, employé de Procter & Gamble (P&G), qui désigne alors le lien entre la technologique RFID et l'Internet.

*Le premier objet connecté est commercialisé en 2003 par la firme Violet. C'est **la lampe DAL**, équipée de 9 leds qui s'allument en fonction des événements. Deux ans plus tard, en 2005, Violet lance le célèbre **lapin Nabaztag**. Racheté par Mindscape puis Aldebaran Robotics en 2013, il disparaîtra en 2015.*

Focus sur l'Internet of Things (IoT)... l'essentiel à savoir
WelcomeToTheJungle



Lampe DAL, Violet, 2003
Sensible au bruit, au toucher, météo, email...



Lapin Nabaztag, Violet, 2005
Diffuse des informations, accès Wifi, lit les emails, la musique, change de couleurs etc

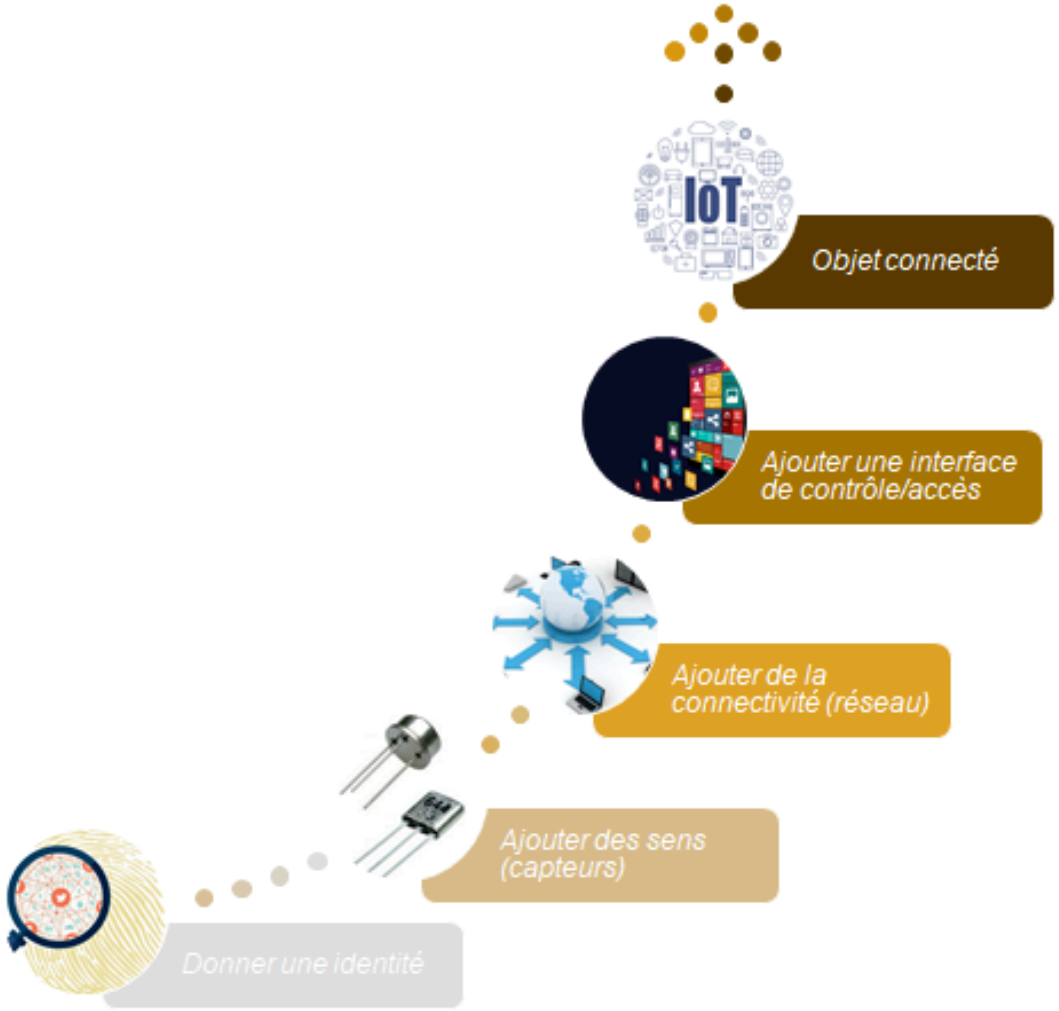
L'avènement de l'IoT et conséquences

Avènement… des façons de connecter des « objets » ?

Comment transformer un objet non connecté en objet connecté ?

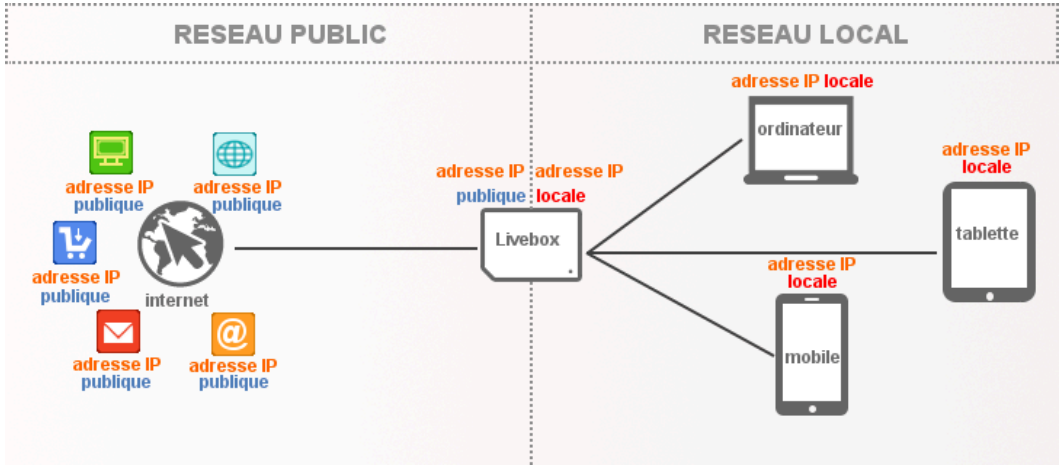
1. Donner une identité (id, adresse)
2. Donner des sens (toucher, vue, ouïe…)
3. Donner un accès au réseau
4. Donner une façon de contrôler/voir les données

Mais... comment connecter et identifier tous ces objets ?!



L'avènement de l'IoT et conséquences

Donner un nom, identifier tous les objets ?



" L'extension de l'espace d'adressage nous permet d'affecter une adresse IPv6 à chaque atome à la surface de la Terre, tout en conservant suffisamment d'adresses en réserve pour 100 planètes Terre supplémentaires.



Steve Leibson

"Guide occasionnel du Musée de l'histoire de l'informatique"

Standardisé par la RFC 82002 en juillet 2017

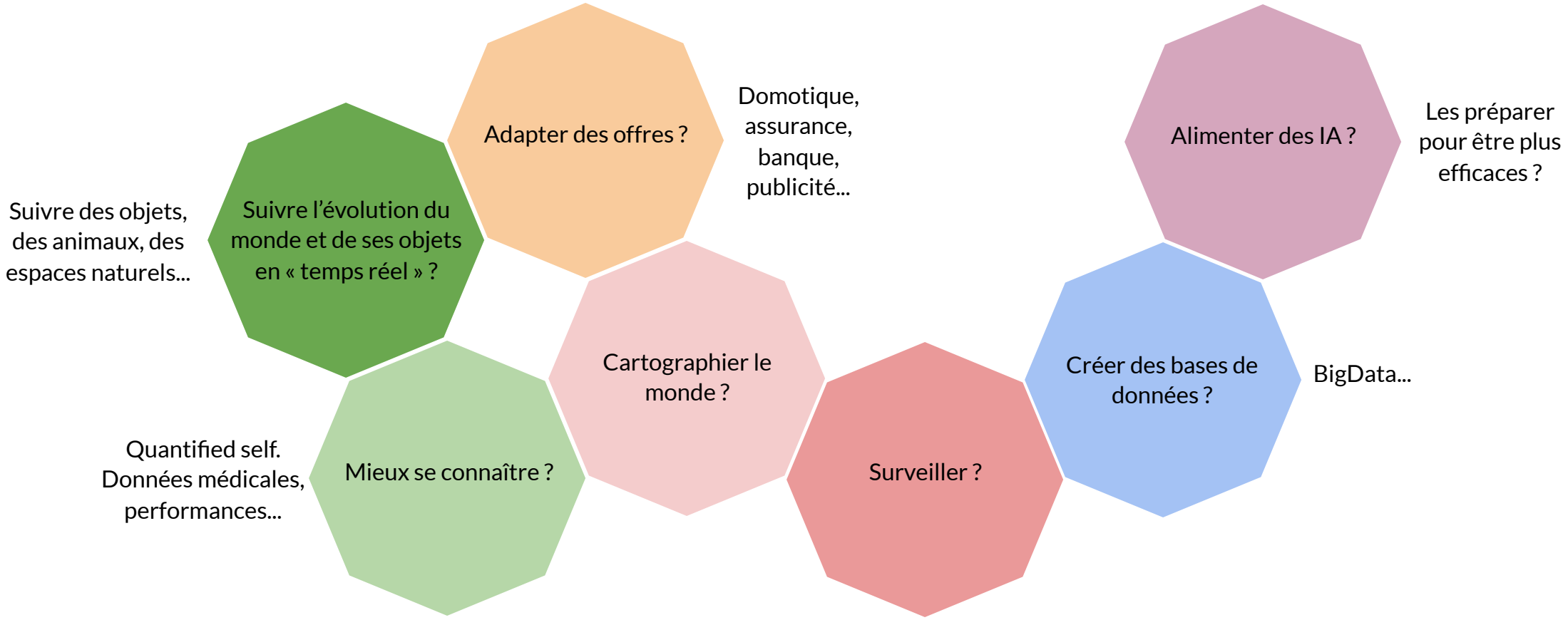
L'avènement de l'IoT et conséquences

Quelques exemples



L'avènement de l'IoT et conséquences

Mais... pourquoi ?



Définitions

Définitions et étapes d'un projet IoT



www.digora.com

Définitions

Définitions et étapes d'un projet IoT

Capteurs / Objets

- Tous les équipements actifs ou passifs pouvant **générer de la donnée exploitable et créatrice de valeur** pour les utilisateurs.
  Les objets sont composés :
  - d'éléments passifs : **les capteurs** (température, humidité, positionnement, temps de fonctionnement, d'alerte),
  - d'éléments actifs les rendant capables **de traitements d'enrichissement de la données et de transmission** de celle-ci.

- Question essentielle : **l'objet disposera-t-il d'une source d'énergie en permanence ?**
  Cela conditionne :
  - L'architecture de la solution,
  - La conception de l'objet,
  - Le cycle de vie de la solution.

Réseau / Connectivité

- **Maillon prépondérant** d'un projet d'IoT, ils doivent répondre à un critère d'usage :
  - La couverture de la zone d'usage des objets (Campus, Ville, Planète…),
  - La consommation énergétique.

D'après DIGORA, devenu Deep.eu

Définitions

Définitions et étapes d'un projet IoT

Données et leur stockage

- **Diamants bruts.** Il s'agit surtout des éléments bruts que nous récoltons depuis les objets ou les outils de process industriels pour l'IoT.
- Afin de créer de la valeur pour les utilisateurs de ces données, il est absolument nécessaire de **les stocker, archiver et sauvegarder** dans des bases de données et de correctement structurer cette dernière.
  En effet **une base de données correctement structurée** améliorera la performance des services IoT d'exploitation.

Informations et leur valeur

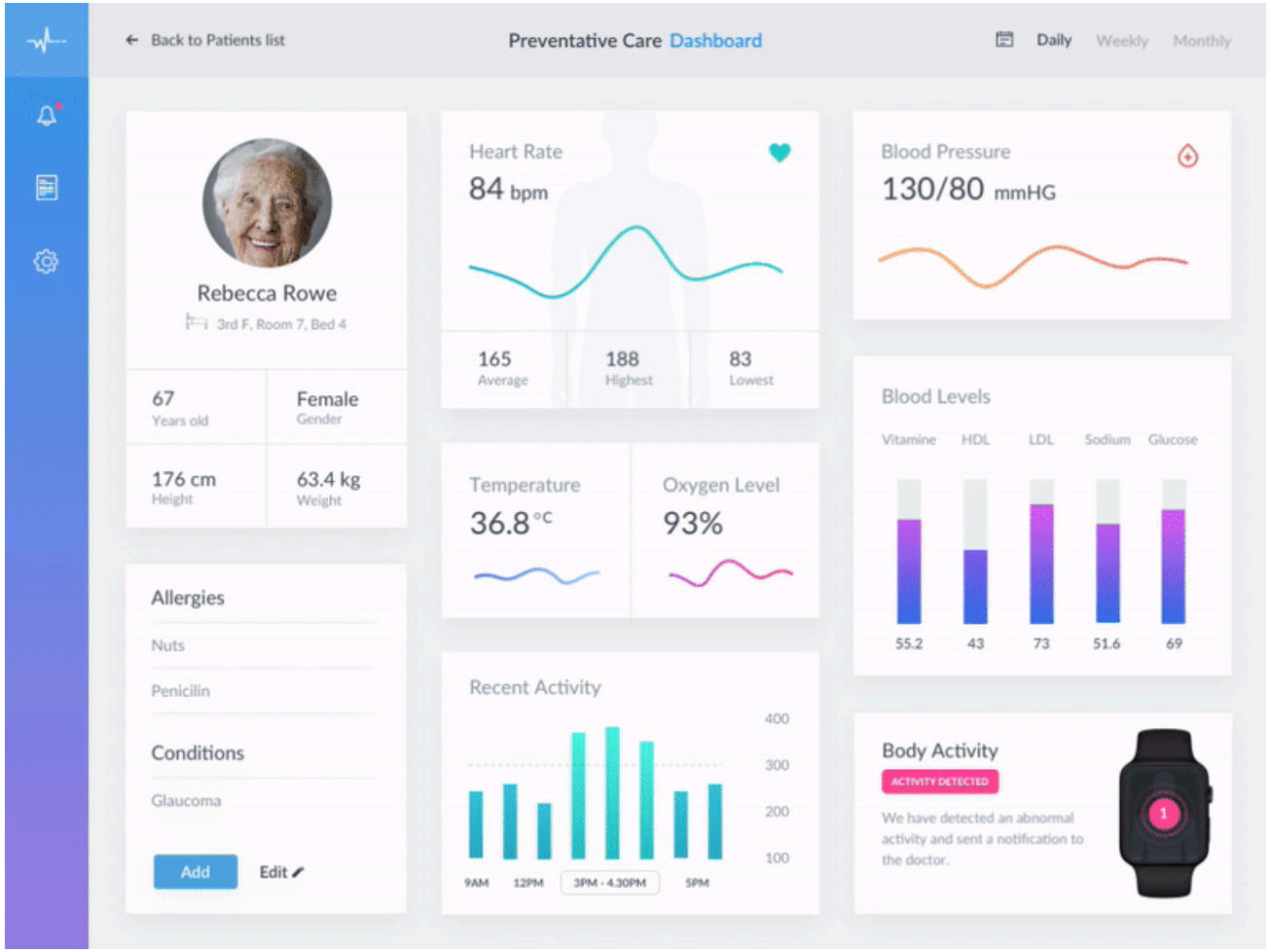
- Résultantes des données traitées, corrélées et analysées, le **Diamant taillé**.
- Ces informations doivent elles-aussi être stockées, archivées et sauvegardées dans des bases de données.

- Un exemple très simple que Digora propose :
  - Le capteur de votre porte d'entrée donne **une donnée** : PORTE OUVERTE OU FERMÉE
  - Nous disposons d'**une donnée universelle** : L'HEURE
  - L'information sera de vous ALERTER : "votre porte est ouverte et à cette heure ce n'est pas normal".
    Nous avons **corrélé deux données**, le capteur de porte et l'heure, **pour générer une information**.

D'après DIGORA, devenu Deep.eu

Définitions

Définitions et étapes d'un projet IoT



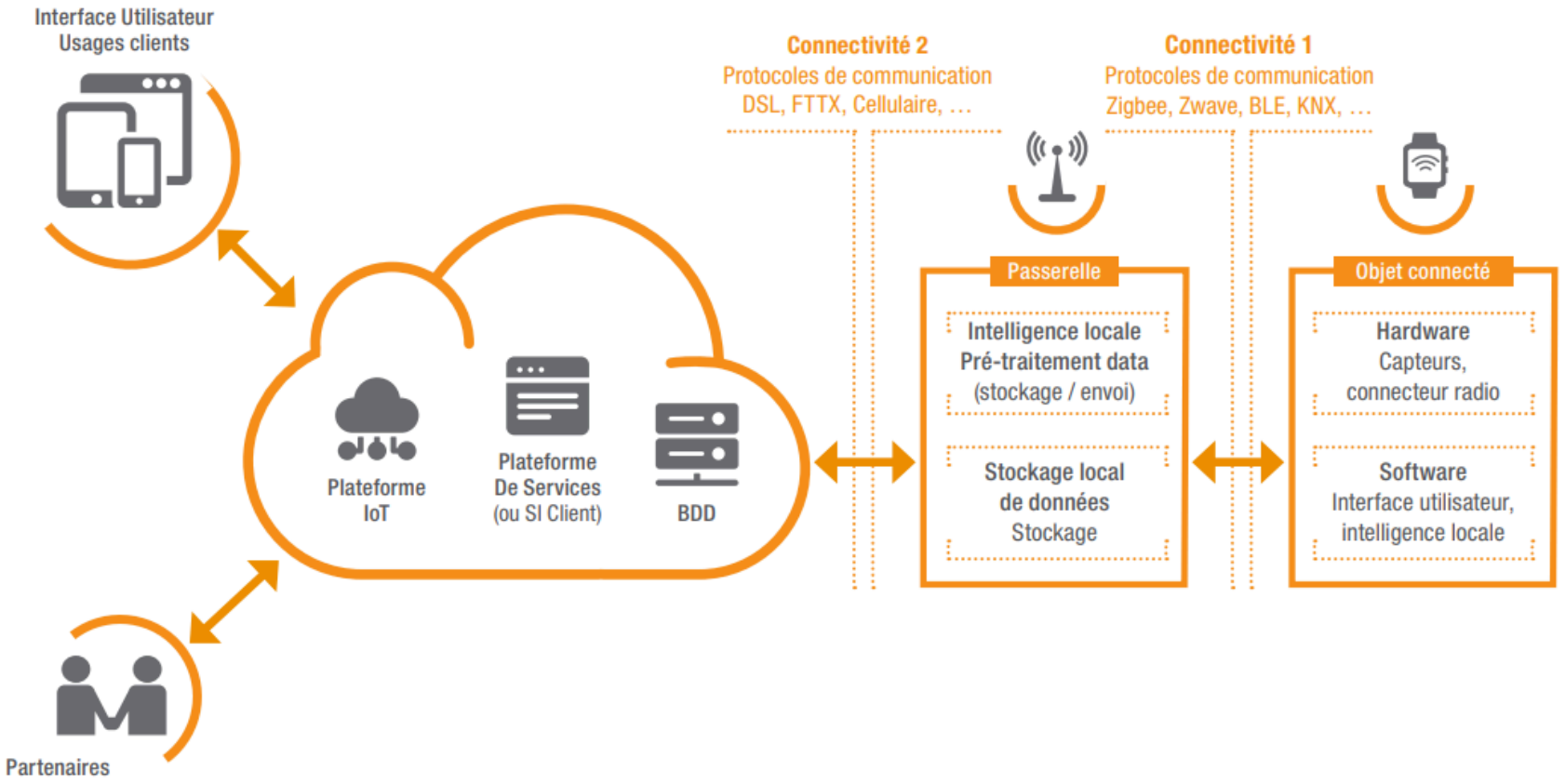
Exploitation pour surveiller et décider

- Les **applications d'exploitation** sont en principe **les interfaces Homme-machine (IHM)** dans lesquelles nous pouvons visualiser les données sous forme de tableau de bord, il s'agit du Diamant monté en bijou.
- Nous retrouvons des outils tels que des **graphiques, des tableaux de données, des rapports...**

D'après DIGORA, devenu Deep.eu

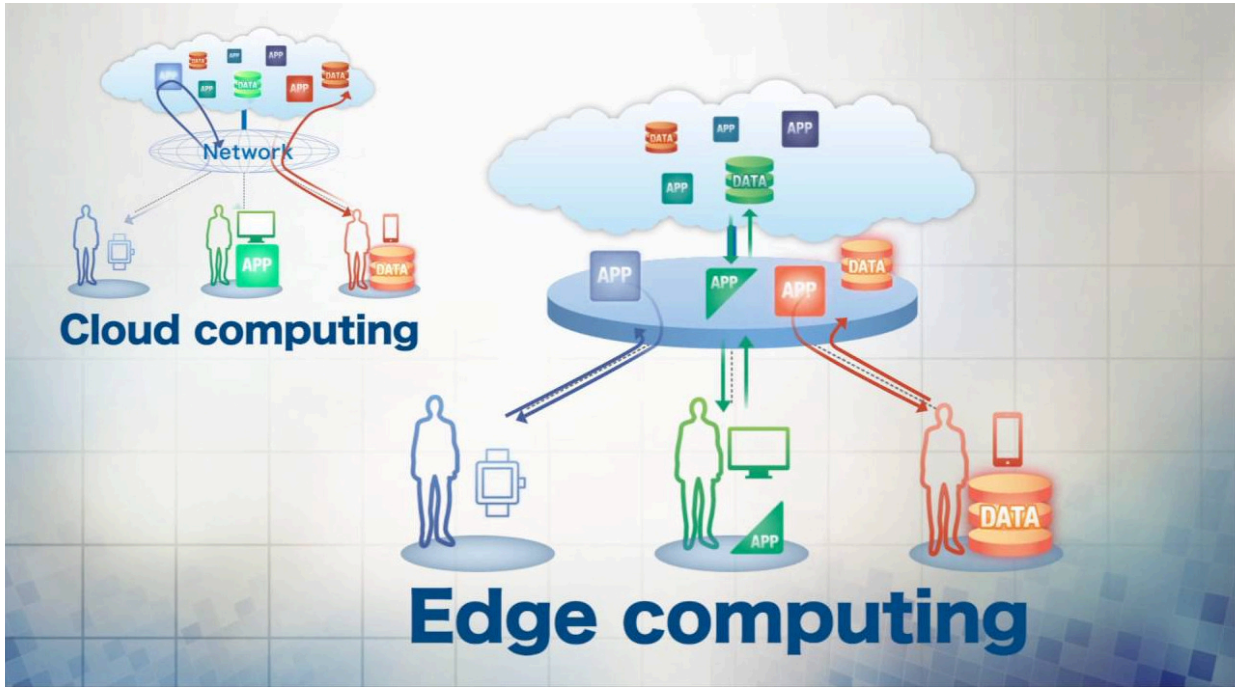
Architectures de projets IoT

Architecture IoT générale



Architectures de projets IoT

Edge Computing : décentraliser le traitement des données



5 avantages, d'après Marc NGIAMBA

1. **Réduction significative de la circulation des données** à travers le réseau. Élimination des goulets d'étranglement résultant de systèmes informatiques centralisés.
   Amélioration de la sécurité des données chiffrées qui restent plus proches de l'utilisateur final en réduisant leur exposition.

2. **Élimination de la dimension centralisée** de l'environnement informatique, réduisant ainsi les points de blocage et de défaillance.

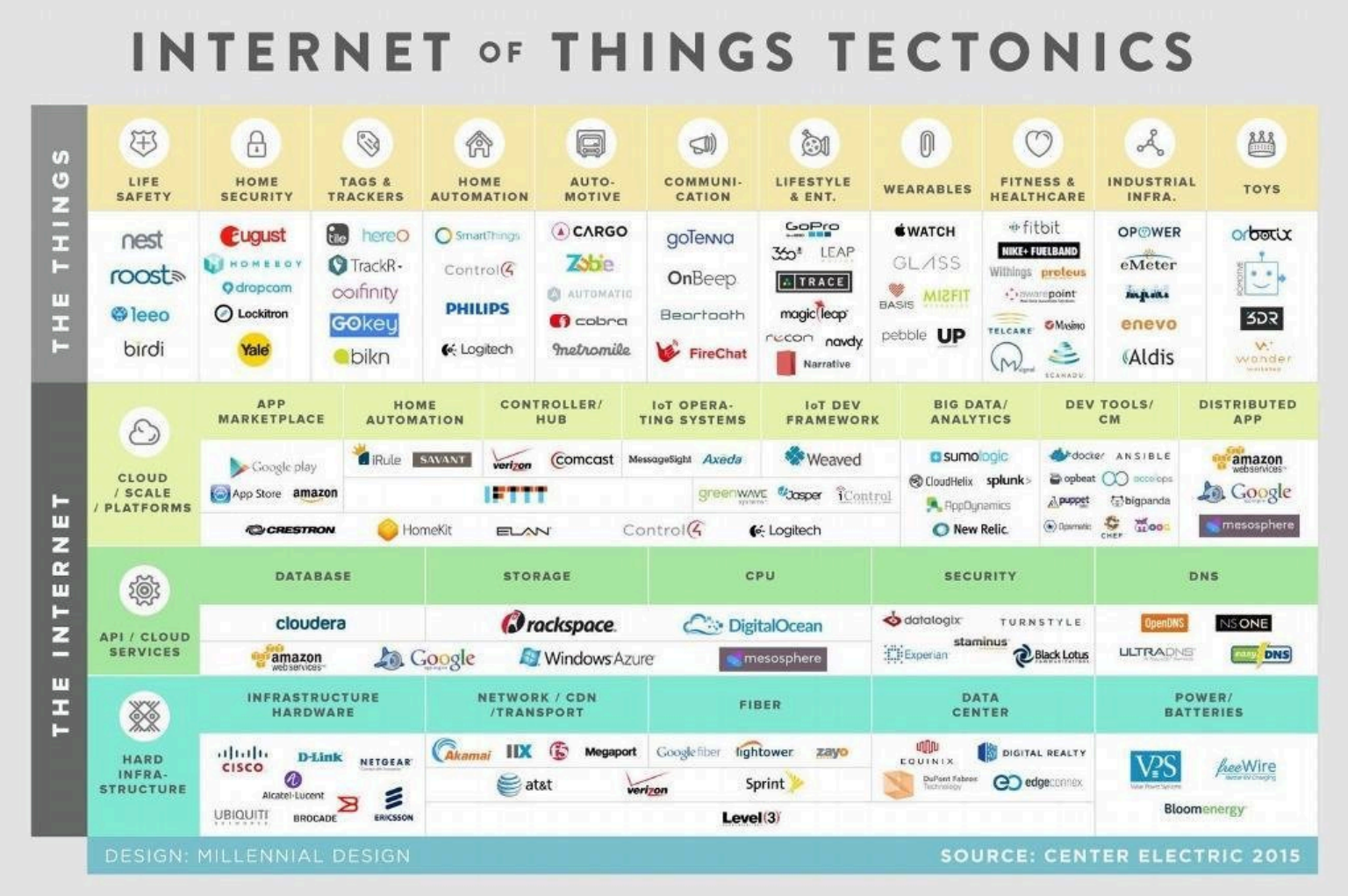
3. **Amélioration de la sécurité**, les données sont codées dès leur déplacement dans le réseau.

4. **Amélioration des temps de réponse aux utilisateurs finaux**. Meilleure évolutivité, fiabilité et tolérance aux pannes.

5. **Consommation réduite de bande passante**.

Architectures de projets IoT

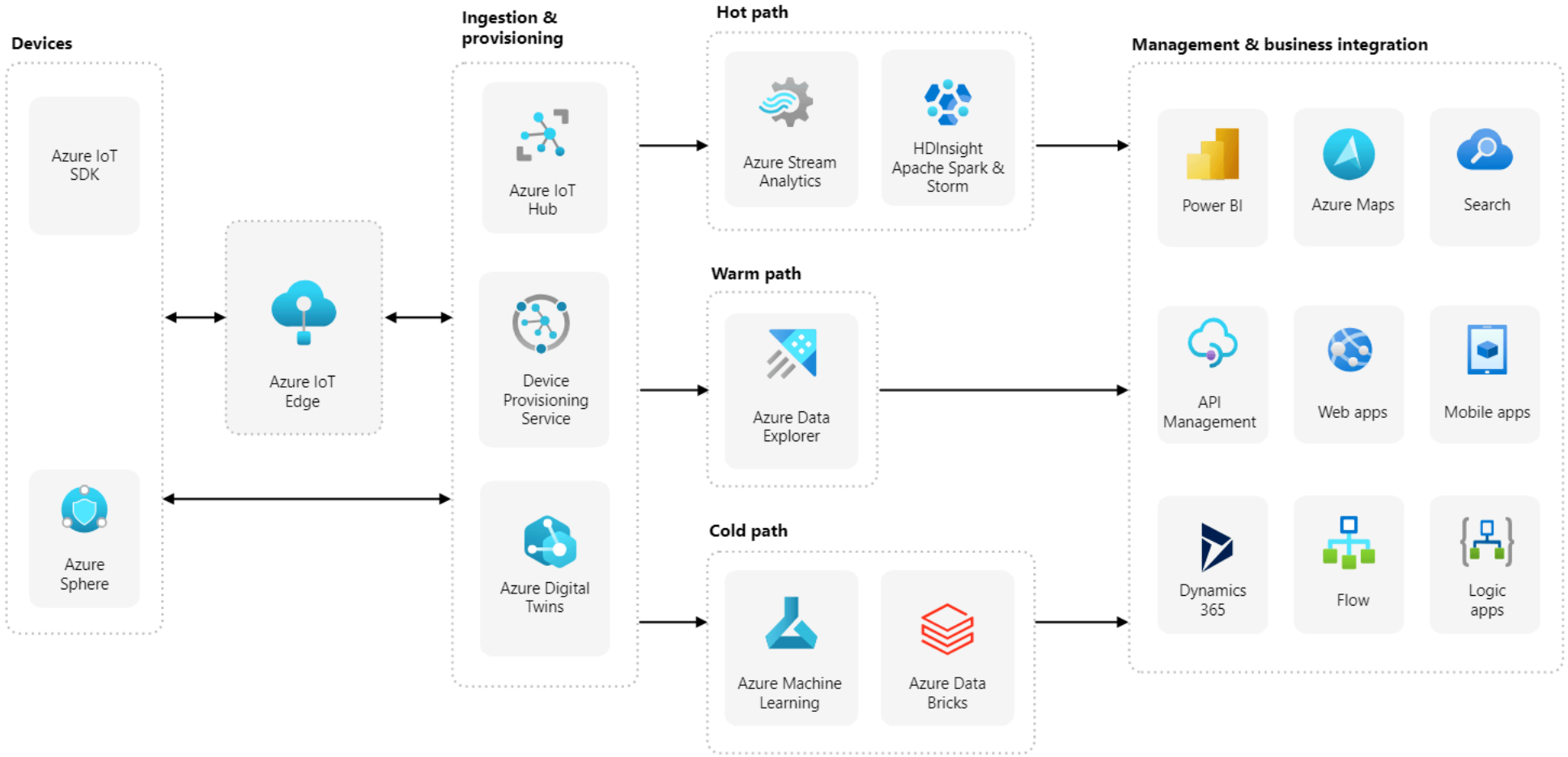
Exemples de solutions existantes



Architectures de projets IoT

Exemples de solutions existantes

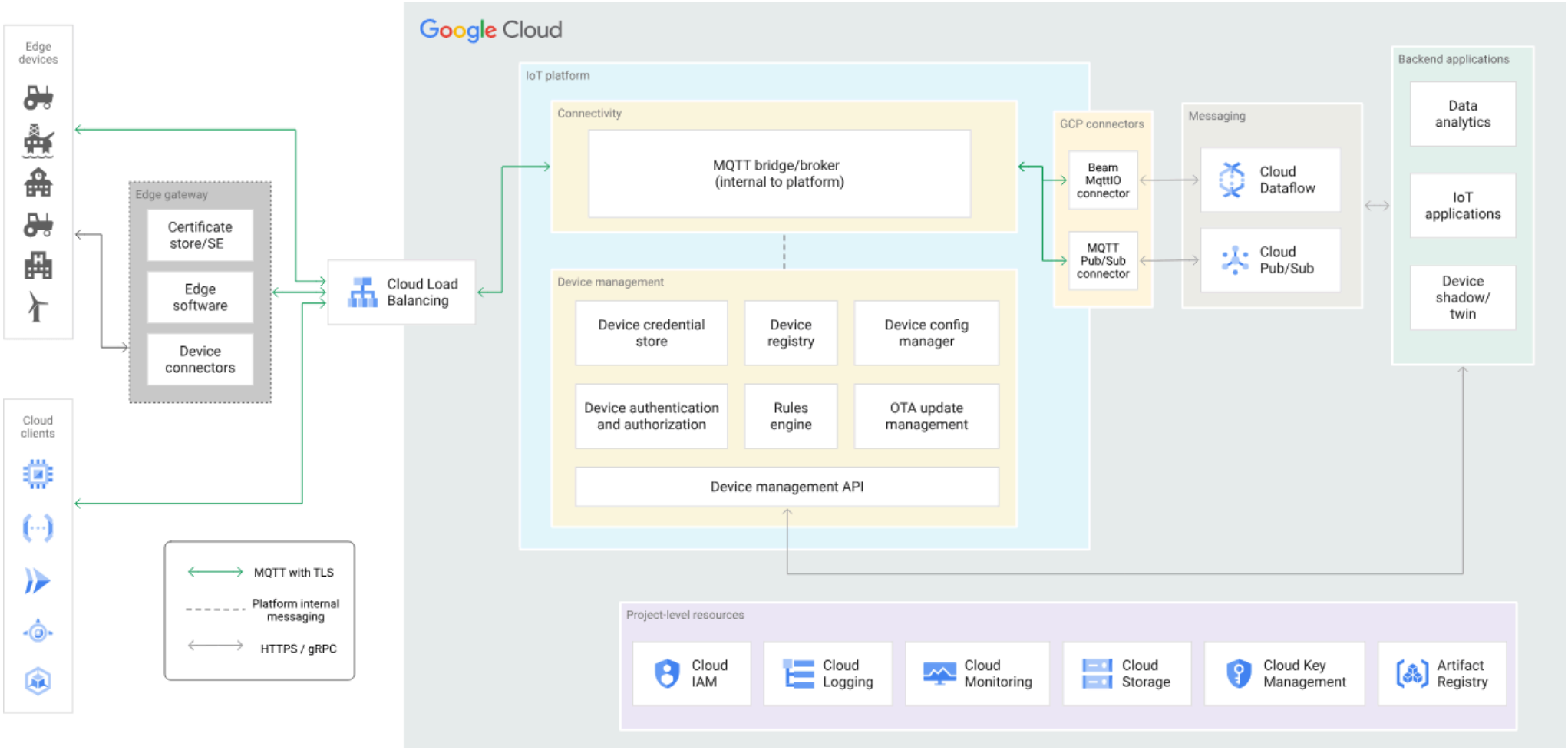
Architecture de référence Azure IoT



Architectures de projets IoT

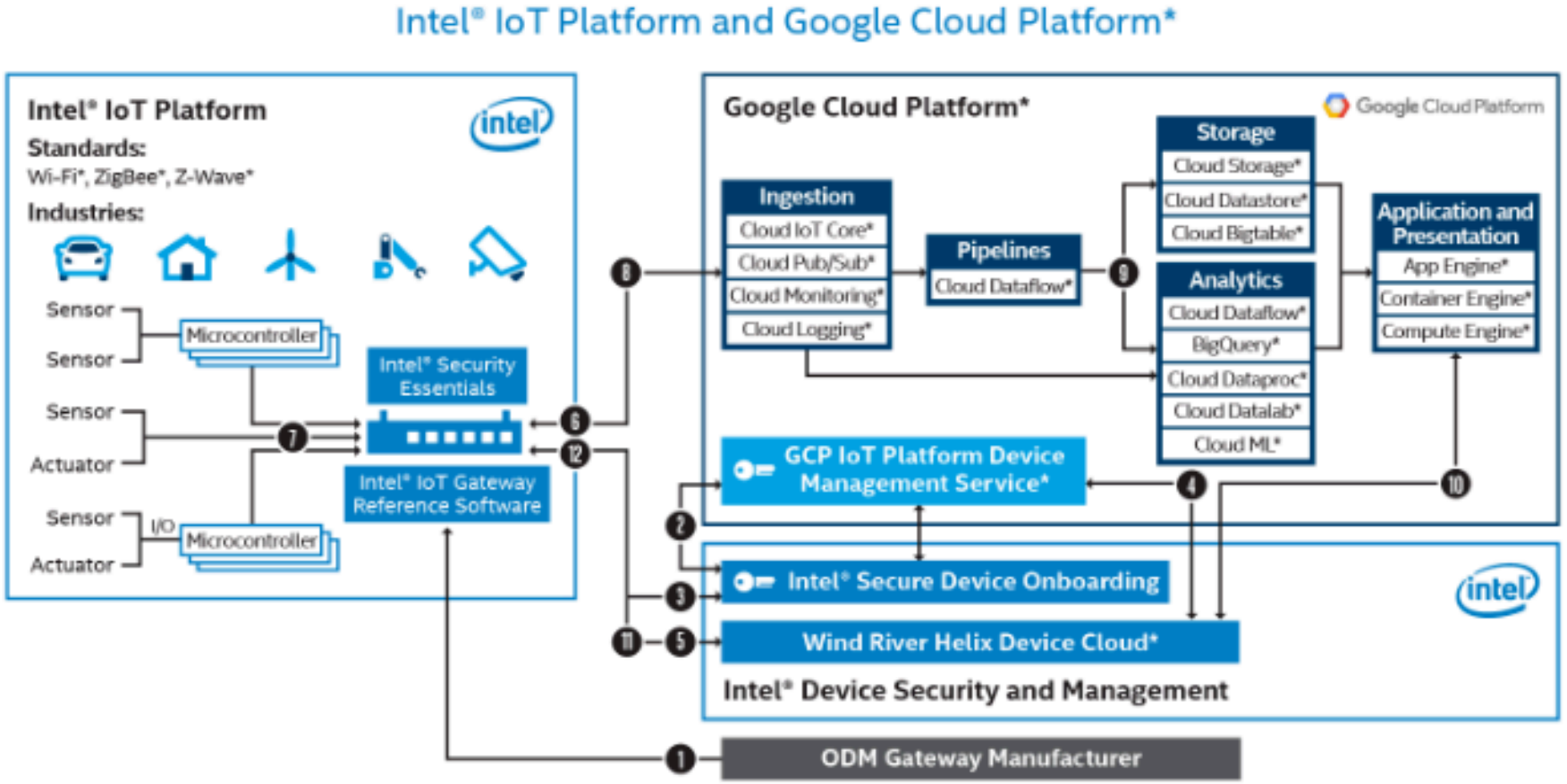
Exemples de solutions existantes

Architecture des produits de plate-forme IoT sur Google Cloud



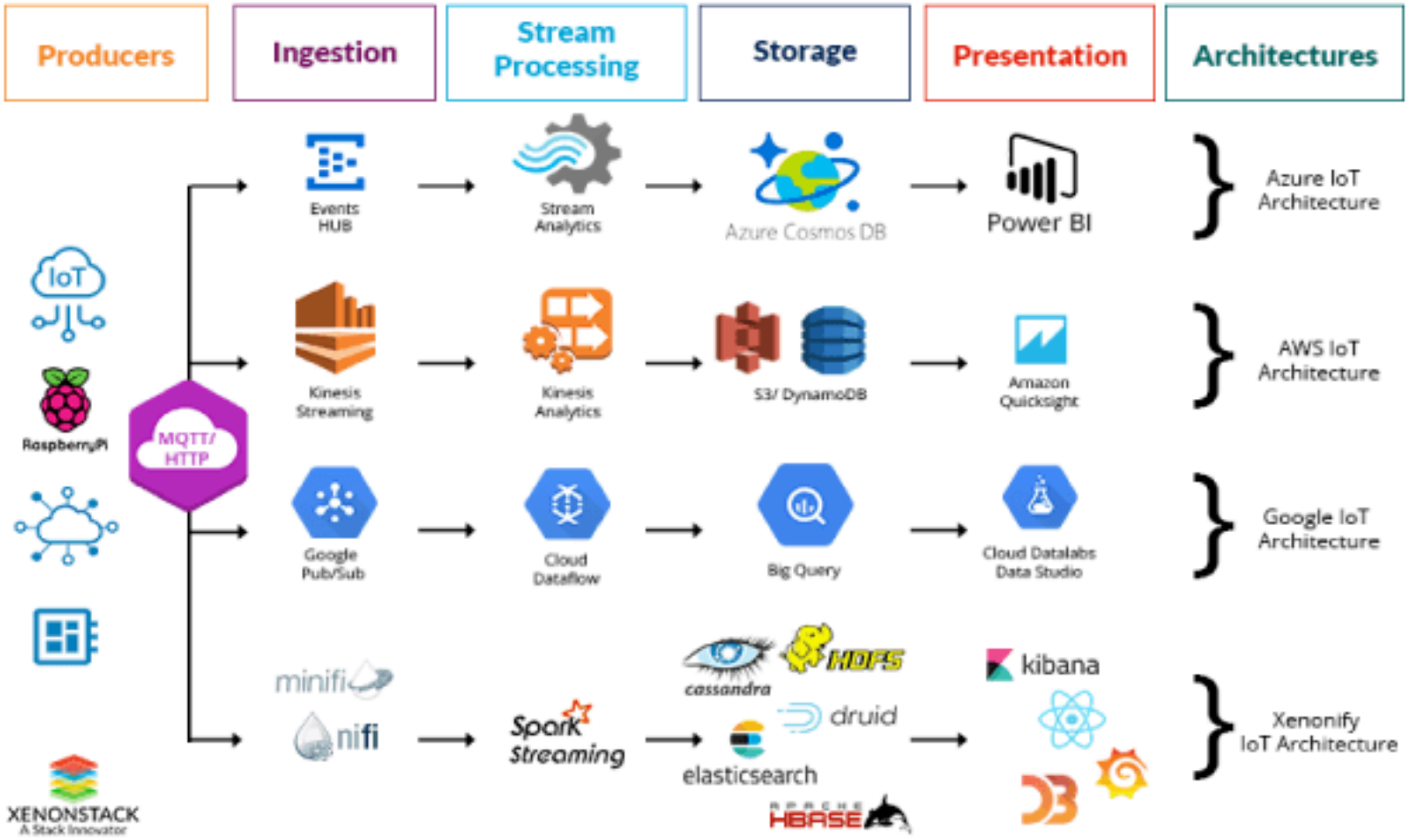
Architectures de projets IoT

Exemples de solutions existantes



Architectures de projets IoT

Exemples de solutions existantes



TP : (re)Découverte de l'ESP32

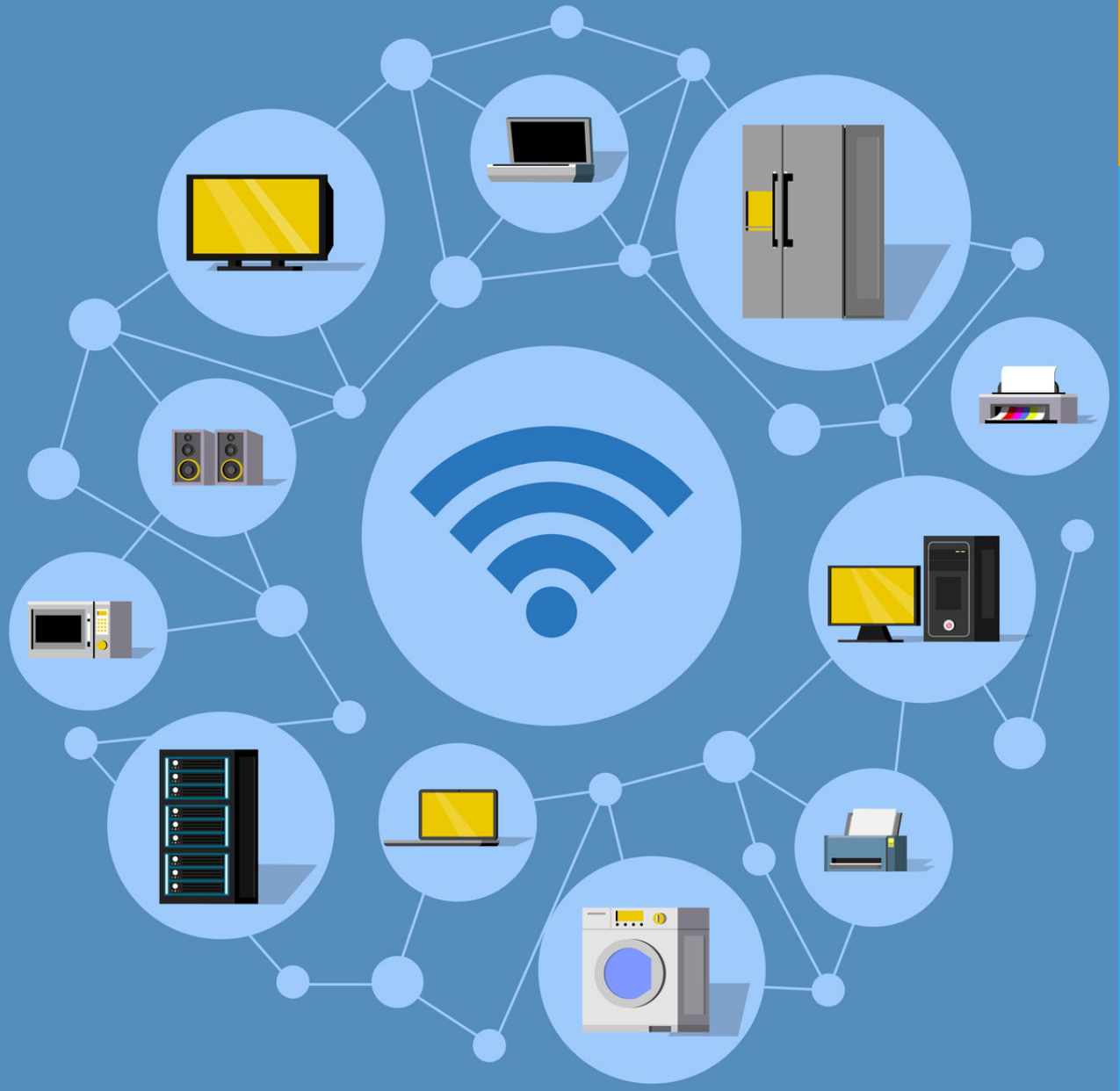
Prise en main de la carte de dev



Man vector created by upklyak - www.freepik.com

Au menu

- Installation IDE et (re)découverte de la plateforme (Qui n'a jamais utilisé une Arduino ?)
- Remémoration des **basiques essentiels**
  - Communication série, UART,
  - Entrée/Sorties numériques, interruptions,
  - Entrée analogique avec calibration,
  - Lecture de datasheet
- Reprise d'**outils nécessaires**
  - I2C et SPI (capteur et afficheur)
  - Connexion WiFi et API
  - Connexion WiFi et Pub/Sub MQTT



Computer vector created by macrovector - www.freepik.com

esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Plan du cours

On va faire quoi en fait ?

1. Introduction Générale

- Déroulé du cours
- (ré)Introduction à l'IoT

2. Objet : plateformes et architectures

- Plateformes et Architectures
- Cas du ESP32

3. Connectivité : réseaux et services Cloud

- **Communication par bus et HAN/LAN/WAN**
- **Principe d'API**
- **Services Cloud**

4. Objet : consommation

- ESP32 et modes de consommations
- Principes de mesure de consommation

5. PoC : à vous de jouer

Communication par bus et HAN/LAN/WAN

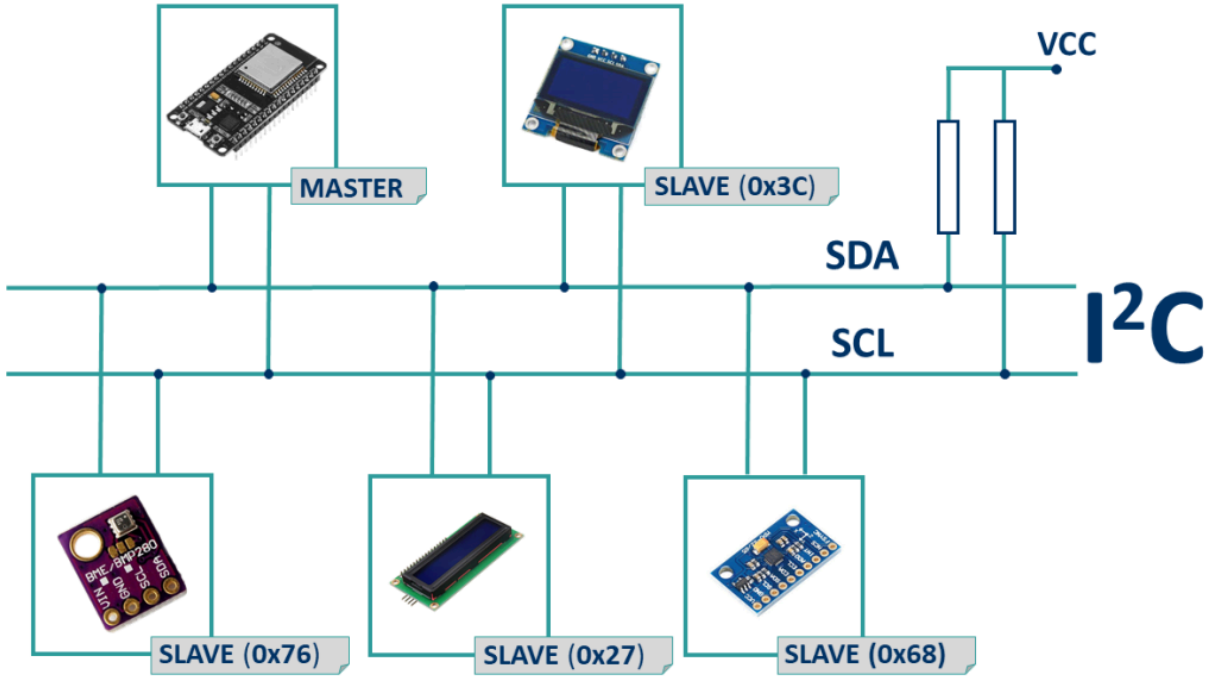
Entre **composants/cartes proches** : communication par bus

Principe de bus de communication

- Reliés par quelques fils
- Utilisation entre carte ou contrôleurs locaux (automobiles, domotique etc)
- Exemples : I²C, SPI, CAN…

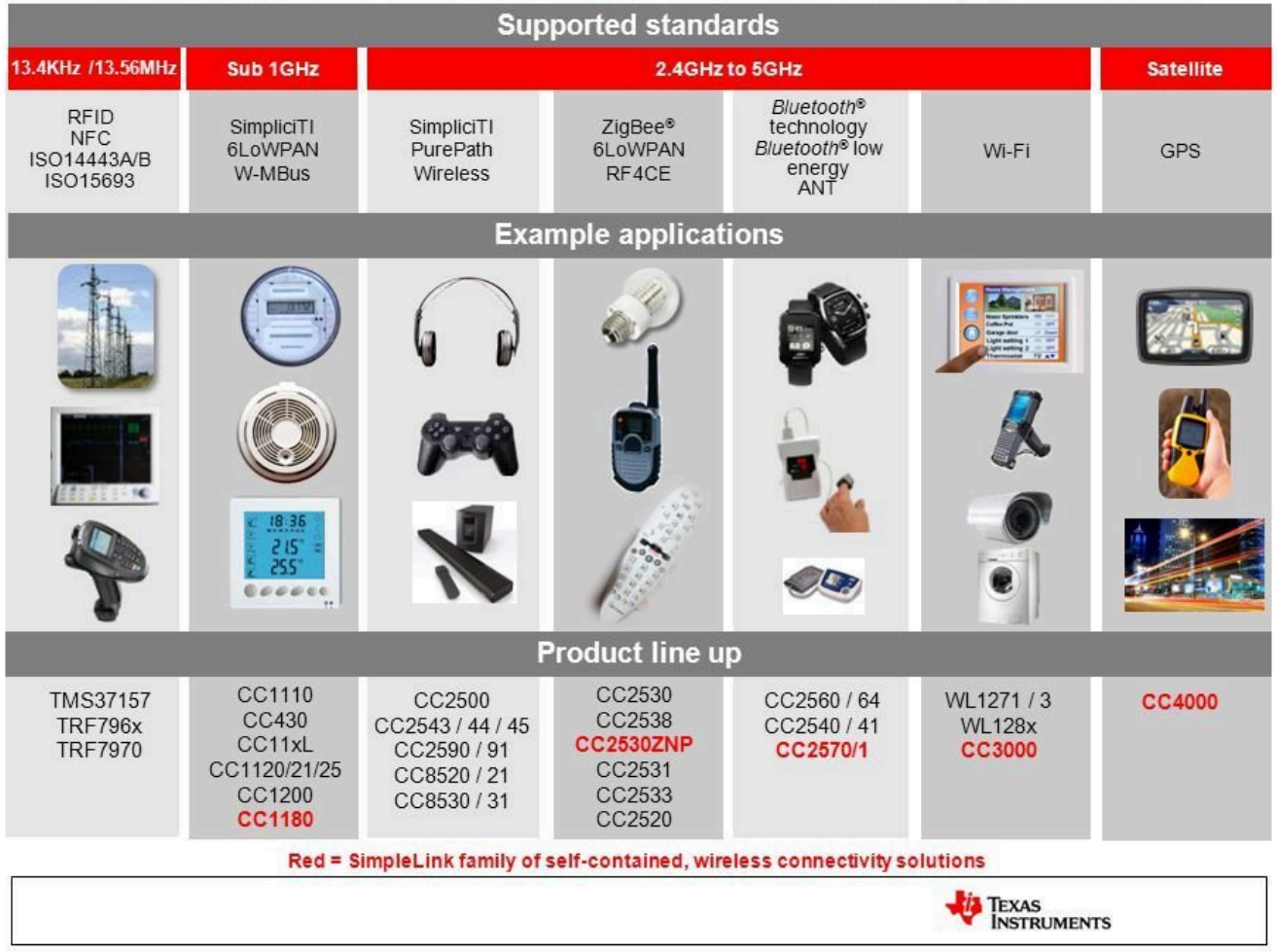
Cas de l'I2C

“ La norme I2C (Inter-Integrated Circuit) a été développée par Philips en 1982. Depuis elle est maintenue par NXP (ex-division semiconducteurs de Philips).
Conçu pour les applications de domotique et d'électronique domestique, il permet de relier facilement un microprocesseur et différents circuits, notamment ceux d'un téléviseur moderne : récepteur de la télécommande, réglages des amplificateurs basses fréquences, tuner, horloge, gestion de la prise péritel, etc.



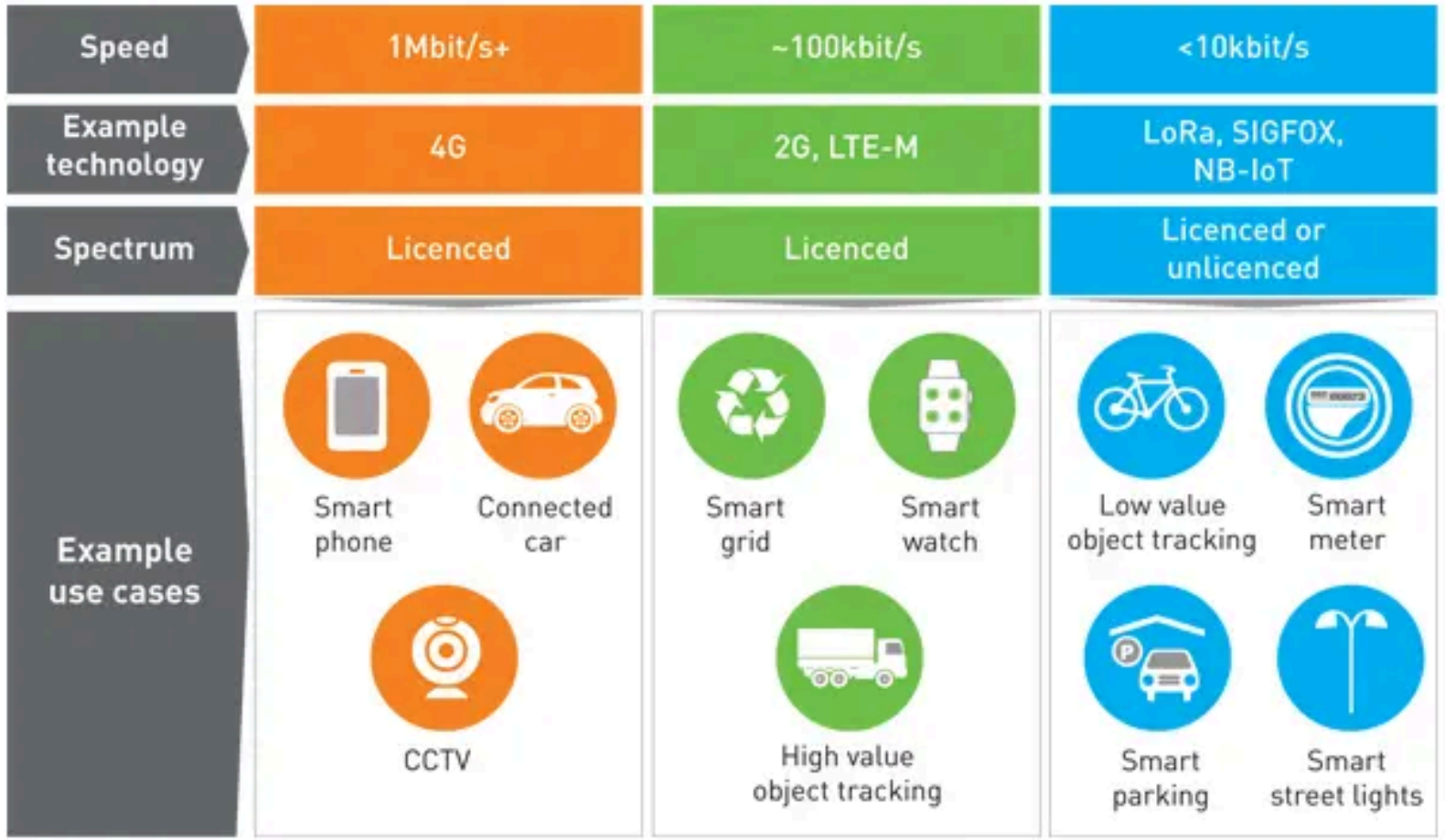
Communication par bus et HAN/LAN/WAN

Entre **objets proches** : Home / Local Area Network (HAN/LAN)



Communication par bus et HAN/LAN/WAN

Entre **objets éloignés** : World Area Network (WAN)



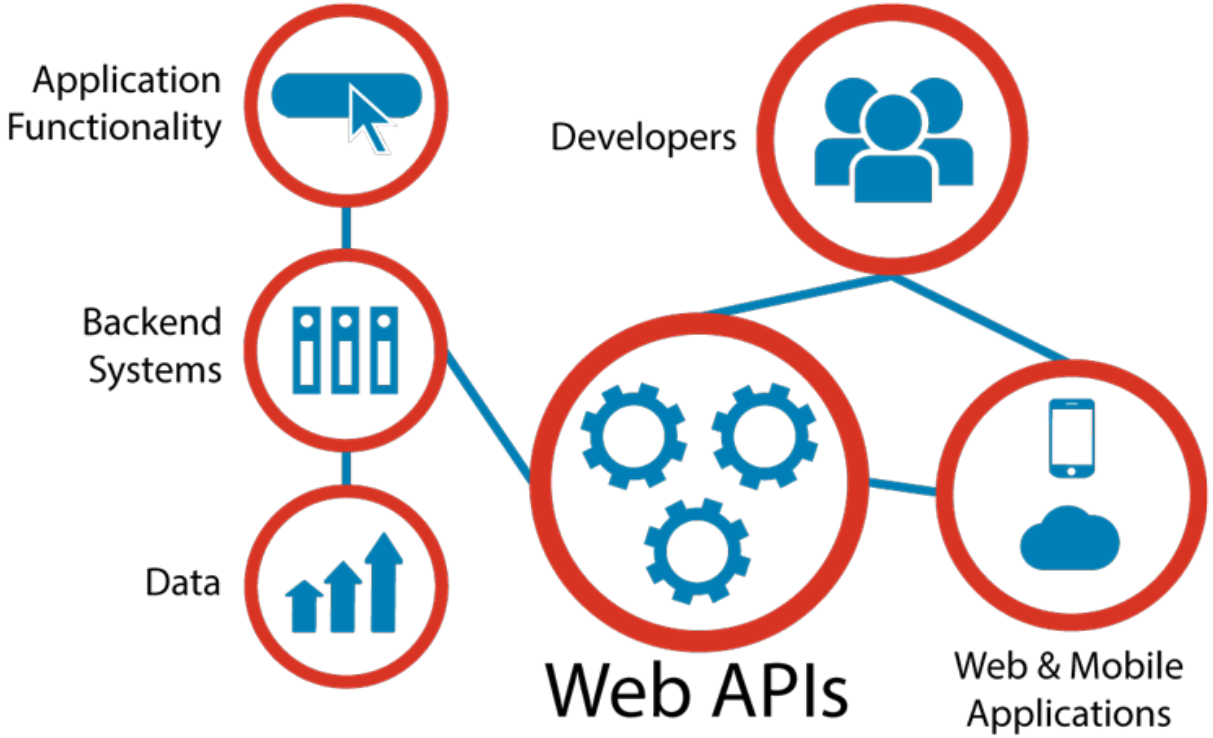
Mapping of use cases and network technologies Source Analysys Mason 2015

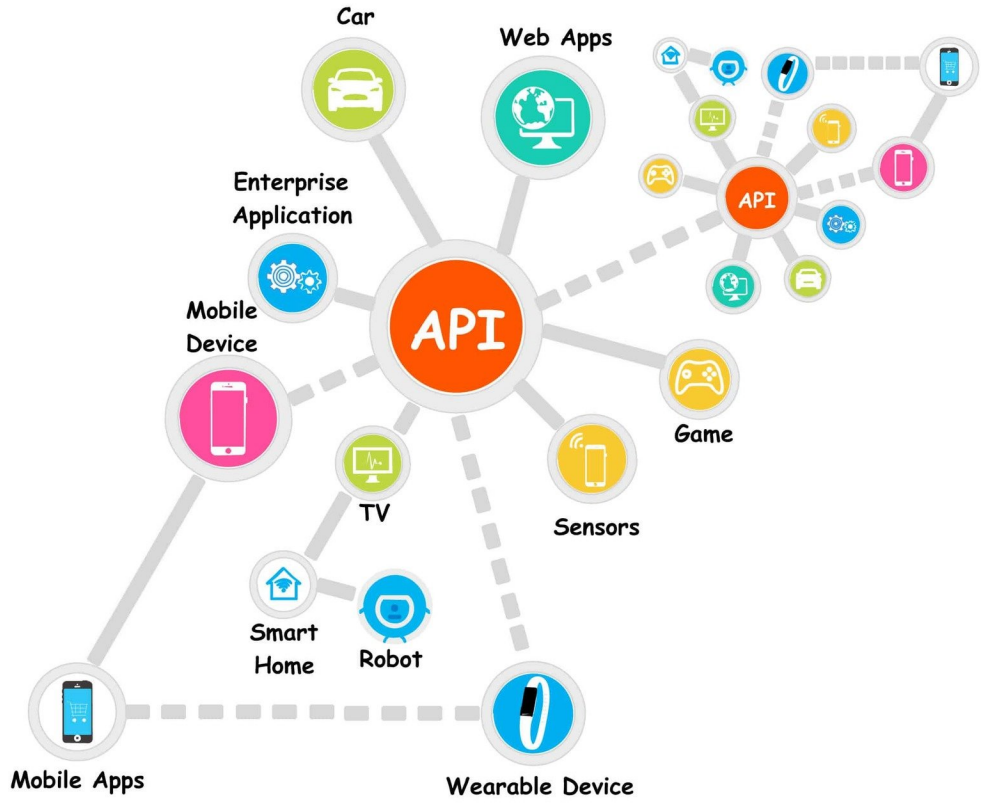
Principe d'API

Une API, c'est quoi ?

“ Une interface de programmation applicative (Application Programming Interface) est une façade clairement délimitée par laquelle un logiciel offre des services à d'autres logiciels. L'objectif est de fournir une porte d'accès à une fonctionnalité en cachant les détails de la mise en œuvre.

API, Wikipedia





Principe d'API

Une API, exemple : Weatherstack (météo)

Key

Après la création d'un compte, on obtient une "**key**", par exemple pour moi : « 14d5901af79657bc0d8cc92341e4e0e4 ». Cette clé permet de m'authentifier, et donc me facturer au nombre de requêtes. Attention, "https" est réservé aux comptes payants.

Requête

- Météo actuelle de Paris : http://api.weatherstack.com/current?access_key=14d5901af79657bc0d8cc92341e4e0e4&query=Paris
- Météo prévue de Marseille : http://api.weatherstack.com/forecast?access_key=14d5901af79657bc0d8cc92341e4e0e4&query=Marseille

Format de réponse

Plusieurs formats structurés utilisés par les API, principalement : XML, JSON

Pour Weatherstack, nous obtenons un "JSON"... que l'on peut rendre plus lisibles...


```
{"request":{"type":"City","query":"Paris,
France","language":"en","unit":"m"},"location":
{"name":"Paris","country":"France","region":"Ile-de-
France","lat":"48.867","lon":"2.333","timezone_id":"Europe\/Paris","localtime
":"2025-08-22
19:02","localtime_epoch":1755889320,"utc_offset":"2.0"},"current":
{"observation_time":"05:02
PM","temperature":21,"weather_code":113,"weather_icons":
["https:\/\/cdn.worldweatheronline.com\/images\/wsymbols01_png_64\/wsymbol_00
01_sunny.png"],"weather_descriptions":["Sunny"],"astro":{"sunrise":"06:53
AM","sunset":"08:53 PM","moonrise":"05:34 AM","moonset":"08:47
PM","moon_phase":"Waning Crescent","moon_illumination":2},"air_quality":
{"co":"344.1","no2":"19.61","o3":"79","so2":"1.295","pm2_5":"8.51","pm10":"17
.205","us-epa-index":"1","gb-defra-
index":"1"},"wind_speed":12,"wind_degree":14,"wind_dir":"NNE","pressure":1020
,"precip":0,"humidity":46,"cloudcover":0,"feelslike":21,"uv_index":1,"visibil
ity":10,"is_day":"yes"}}
```



```
{
  "request": {
    "type": "City",
    "query": "Paris, France",
    "language": "en",
    "unit": "m"
  },
  "location": {
    "name": "Paris",
    "country": "France",
    "region": "Ile-de-France",
    "lat": "48.867",
    "lon": "2.333",
    "timezone_id": "Europe/Paris",
    "localtime": "2025-08-22 19:02",
    "localtime_epoch": 1755889320,
    "utc_offset": "2.0"
  },
  "current": {
    "observation_time": "05:02 PM",
    "temperature": 21,
    "weather_code": 113,
    "weather_icons": [
```

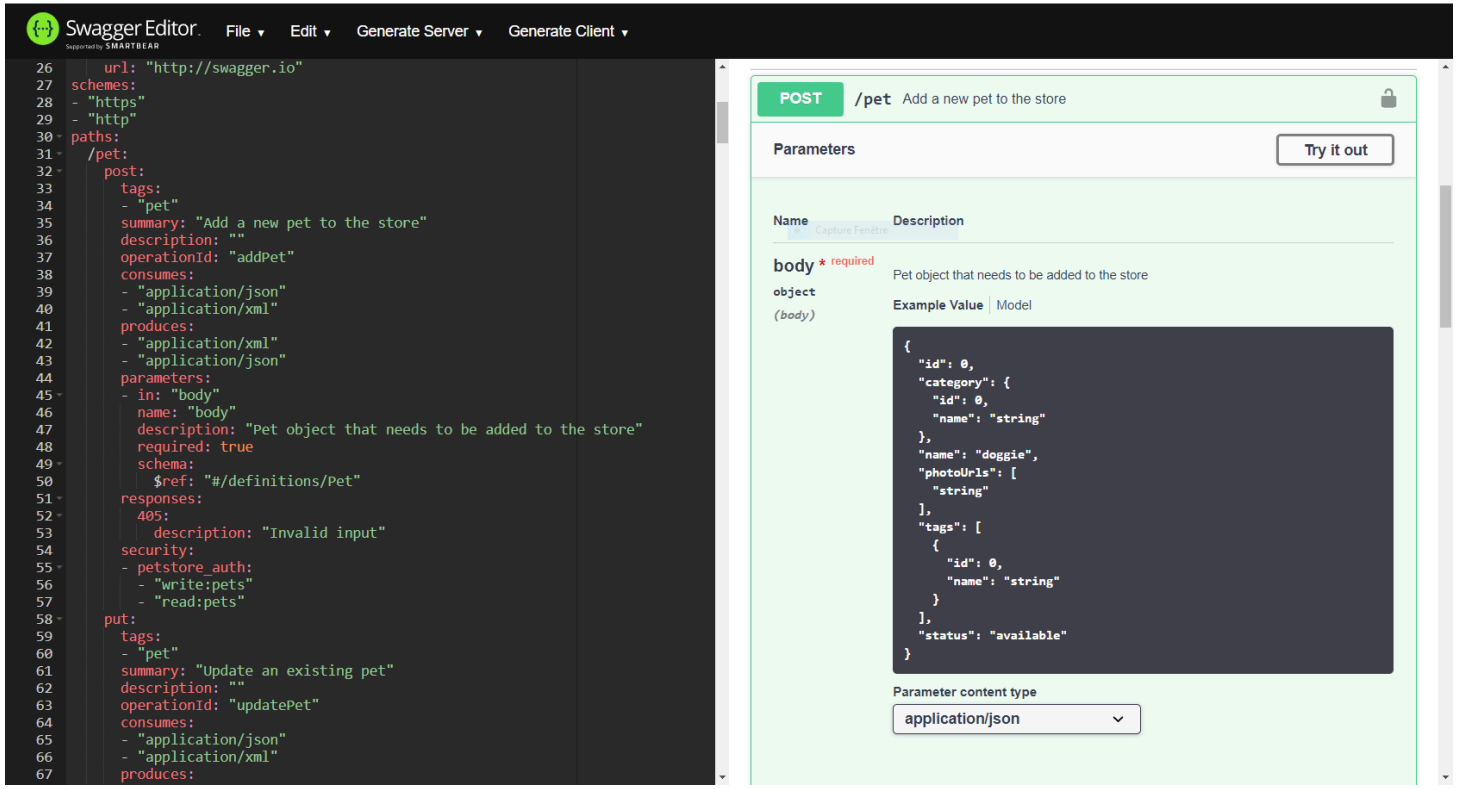
Principe d'API

OpenAPI / Swagger : un allié de choix

Pourquoi OpenAPI Specification ?

*“ The OpenAPI Specification, formerly known as the Swagger Specification, is the world’s standard for defining RESTful interfaces. The OAS enables developers to design a **technology-agnostic API interface** that forms the basis of their API development and consumption.*

Swagger.io

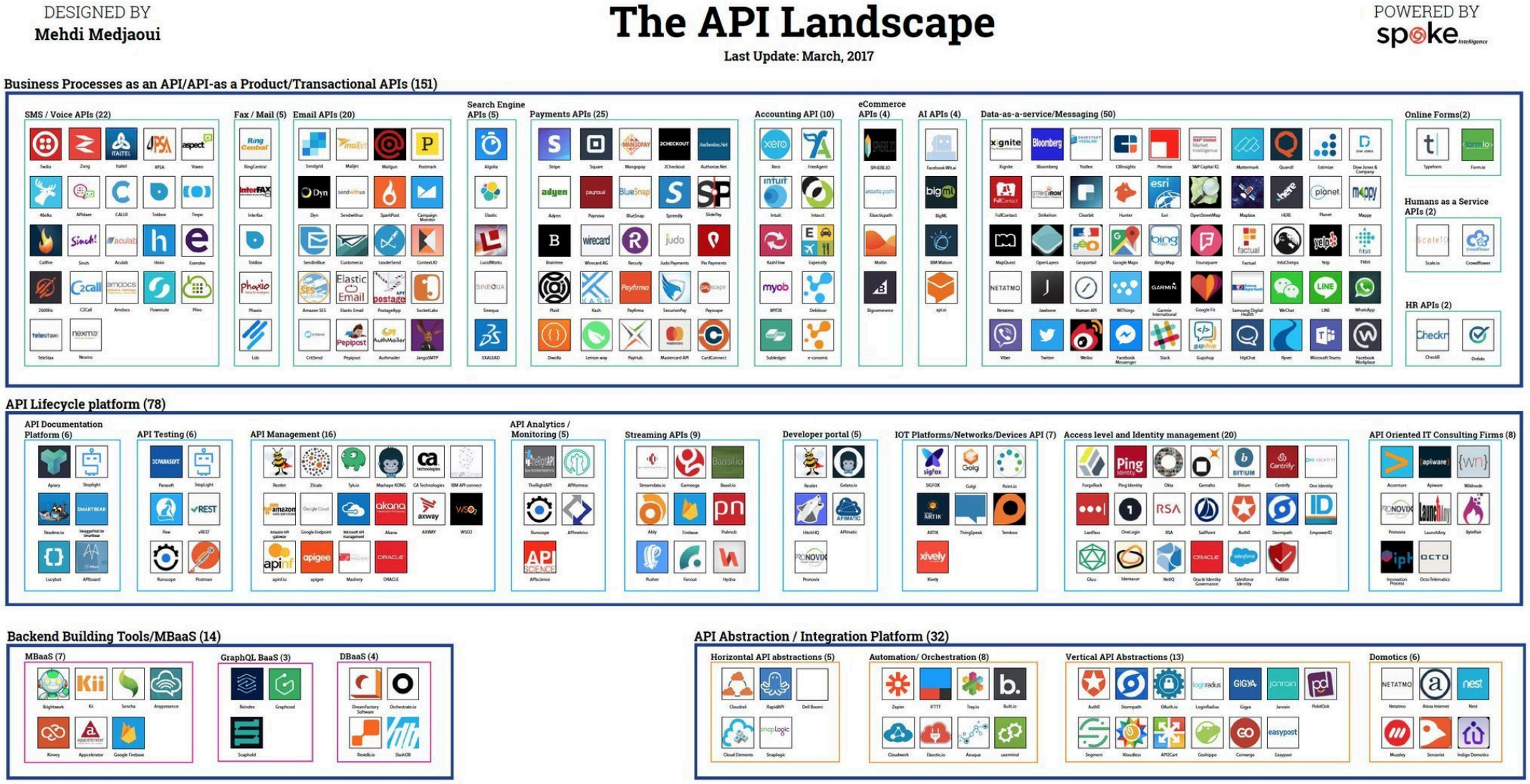


Swagger OpenSource

- Swagger Editor
  - Ecrire la documentation de l'API en OpenAPI et avoir un retour visuel direct.
- Swagger UI
  - Accéder à la documentation mise en forme afin de savoir comment utiliser une API
- Swagger CodeGen
  - Générer un code de base à partir d'une définition OpenAPI, que ce soit le serveur ou le client dans votre langage de programmation préféré.

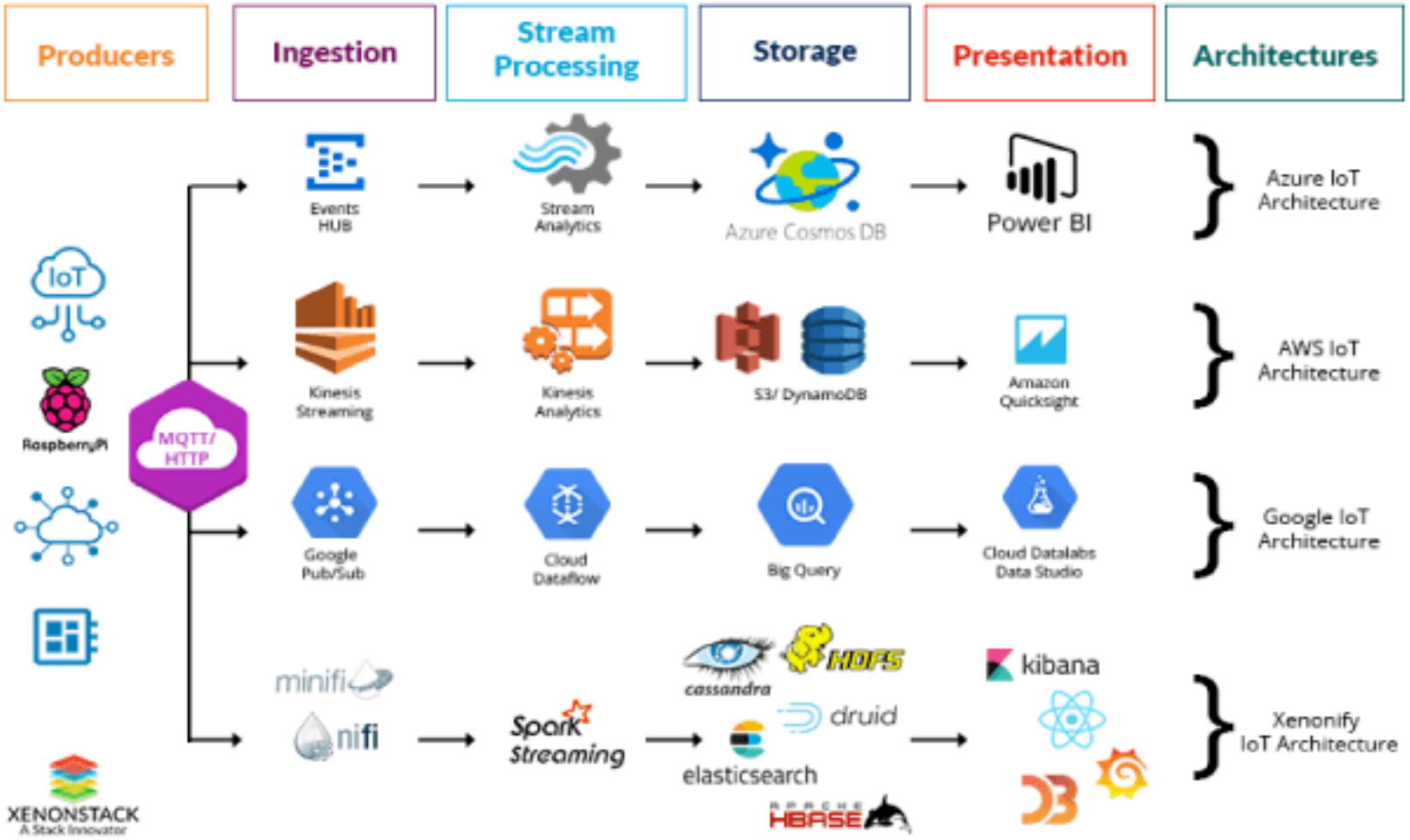
Principe d'API

Exemple



Services Cloud

Exemples de solutions existantes



Services Cloud

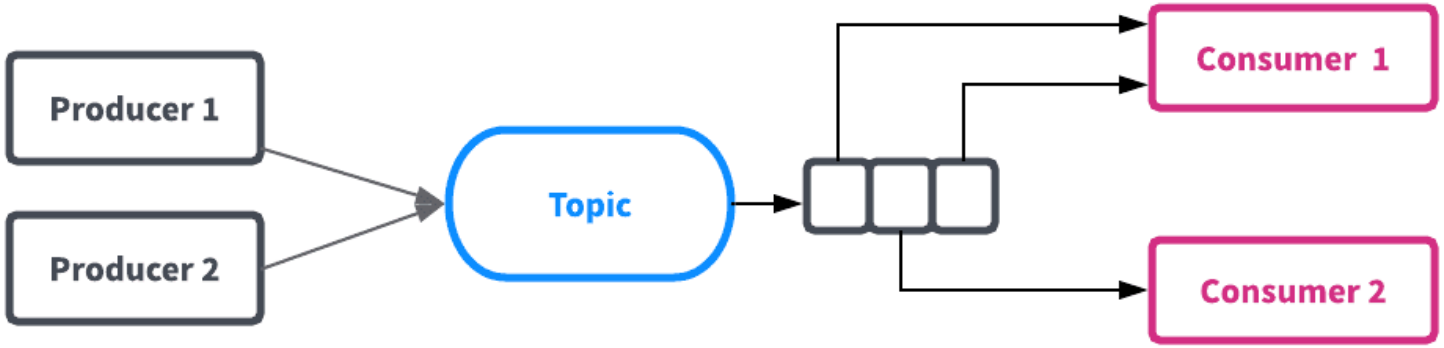
Focus Ingestion et DataRouting

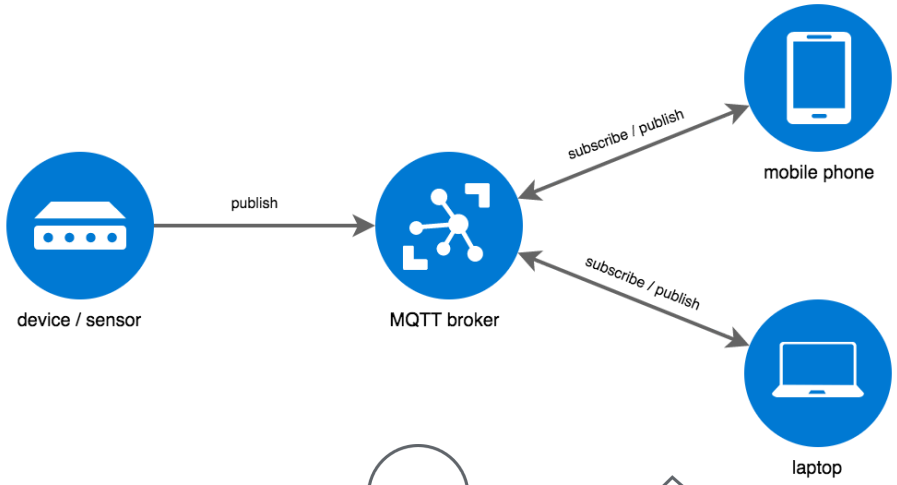
Les données arrivent :

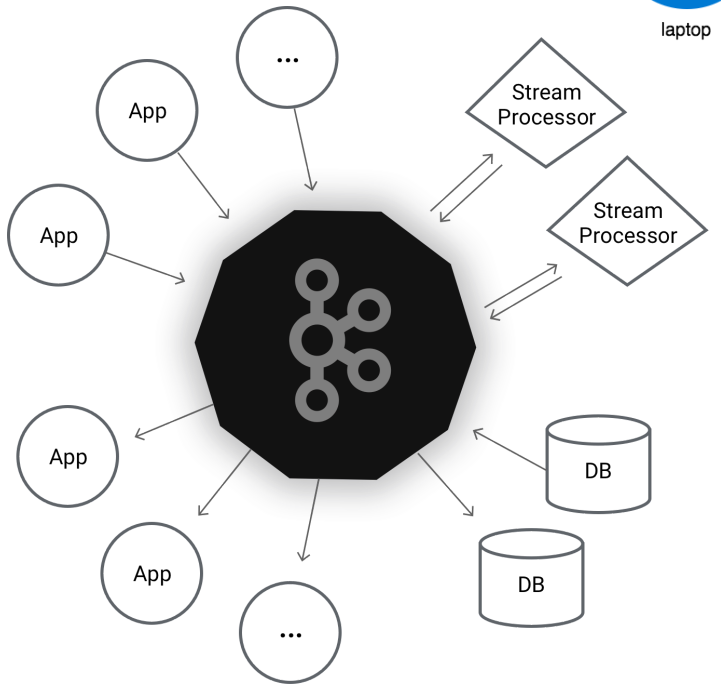
- A des fréquences différentes (donc : n'importe quand)
- Dans des formes différentes (donc : n'importe comment)
- Par des moyens/réseaux différents (donc : de n'importe où)

Un principe : « IoT Hub » / « Message Queuing » / « Message Broker »
==> Principe de Topics et PUB/SUB

- Locaux/Objet : MQTT (Mosquitto)
- Cloud : AMQP (RabbitMQ, Apache Kafka etc)
- Interchangeables dans certains contextes...









esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Plan du cours

On va faire quoi en fait ?

1. Introduction Générale

- Déroulé du cours
- (ré)Introduction à l'IoT

2. Objet : plateformes et architectures

- Plateformes et Architectures
- Cas du ESP32

3. Connectivité : réseaux et services Cloud

- Communication par bus et HAN/LAN/WAN
- Principe d'API
- Services Cloud

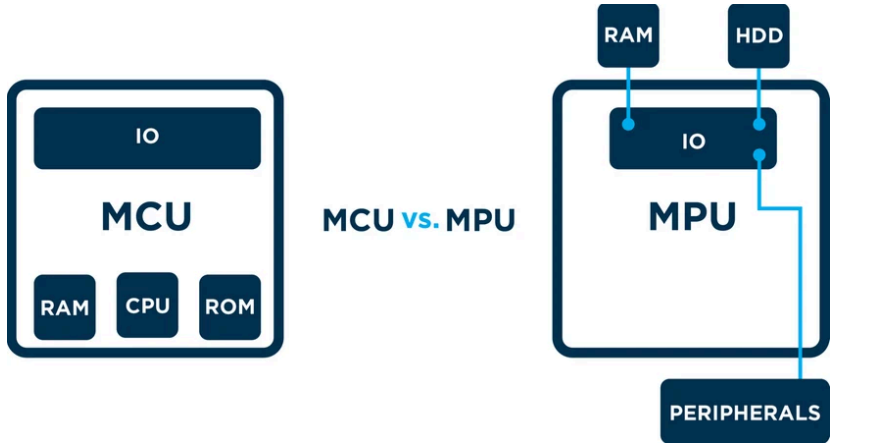
4. Objet : consommation

- **ESP32 et modes de consommations**
- **Principes de mesure de consommation**

5. PoC : à vous de jouer

Plateformes et Architectures

Microprocesseur (MPU) ou Microcontrôleur (MCU) ?



Plus petit et intégré
Plus économe en énergie

Moins puissant
Moins adaptable

Plus adaptable et évolutif
Plus puissant

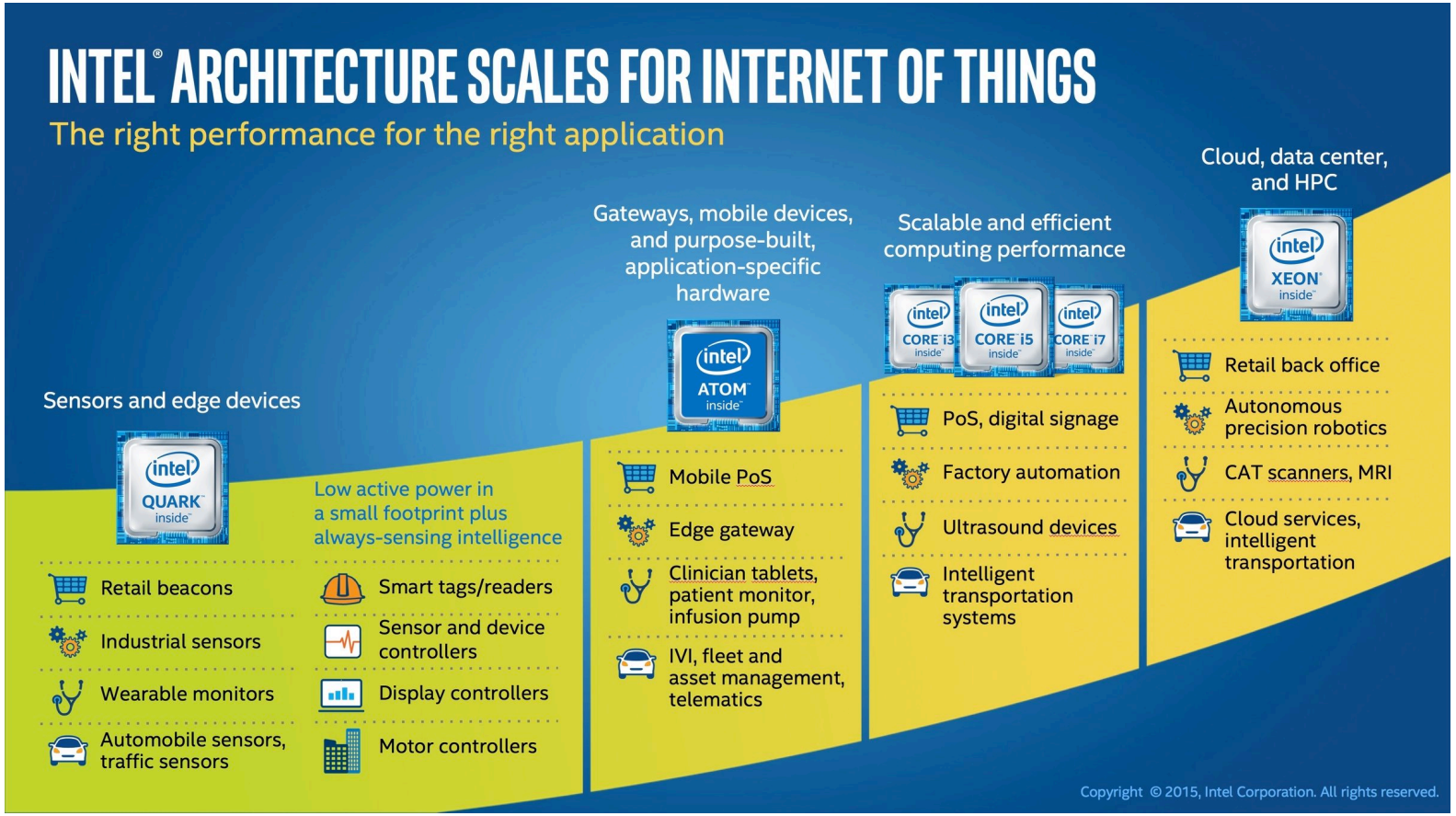
Prend plus de place
Consomme d'avantage

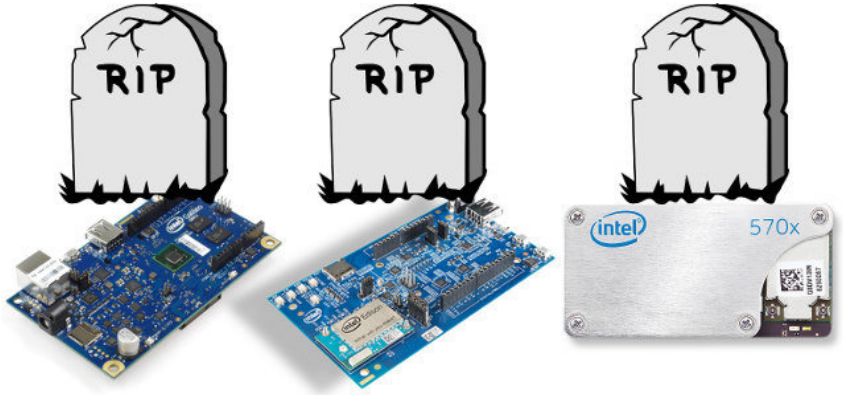
Ou même…

MPU	SoC	MCU	DSP	FPGA/ASIC
Only CPU	CPU, GPU, Radio, HW accelerators	CPU, I/O	CPU, I/O, special maths functions	Hardware functions
	Complex devices (computers, TVs,…)	Low power devices (battery-powered)	Signal processing (music, video, radars,…)	Signal processing // computing integrated circuit prototyping
	Expensive	Cheap	Wide range of prices	Expensive
With or without OS	OS strongly recommended	With or without OS	With or without OS	With or without OS

Plateformes et Architectures

MPU, Intel ?

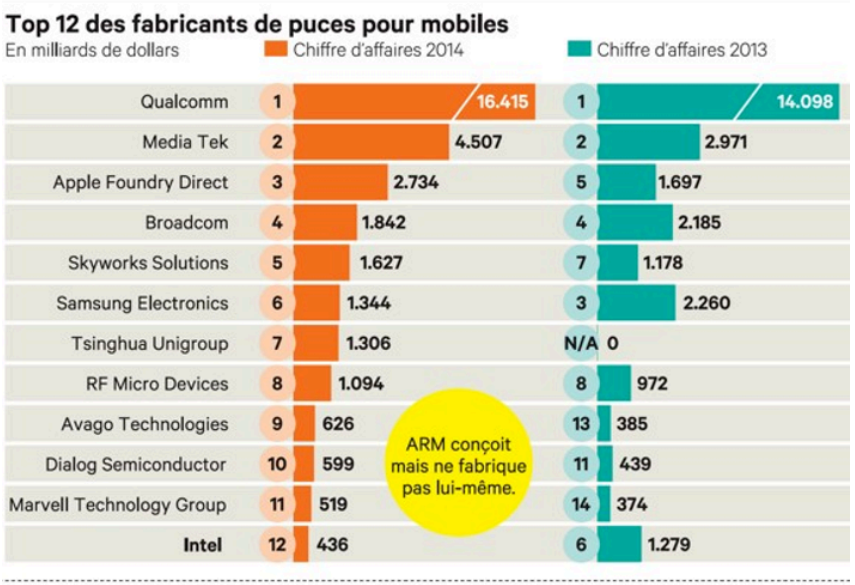




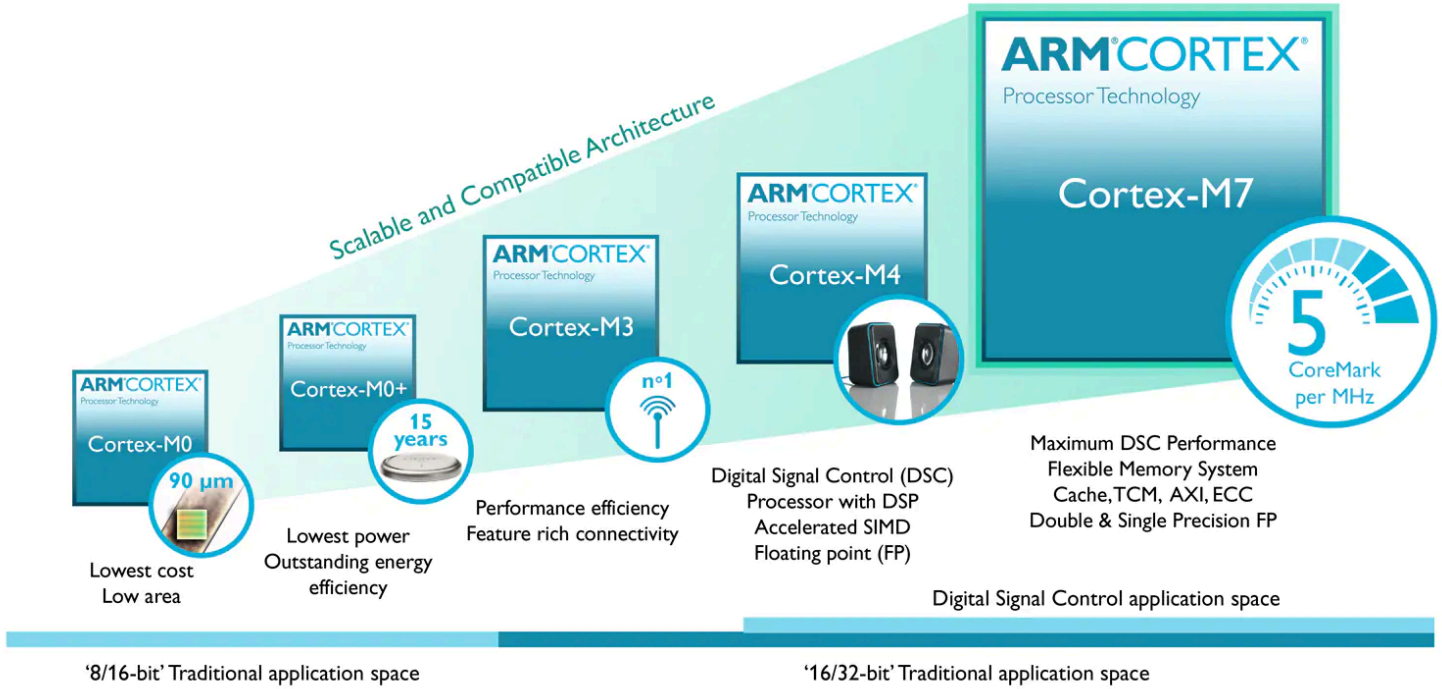
Intel Galileo Board (2013-2017)
Intel Edison (2014-2017)
Intel Joule 550x et 570x (2016-2017)

Plateformes et Architectures

MPU, ARM ? C'est qui ?

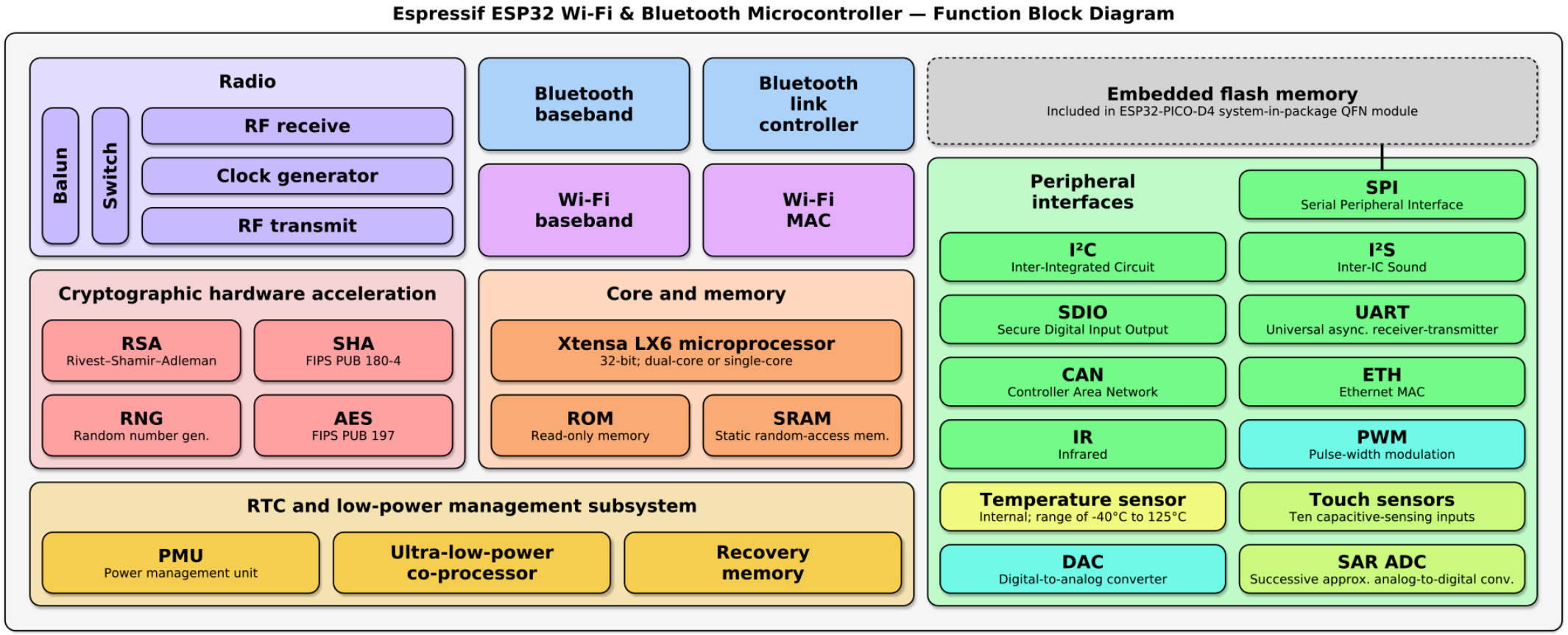


Nvidia veut acheter ARM en 2020...
et abandonne en 2022...



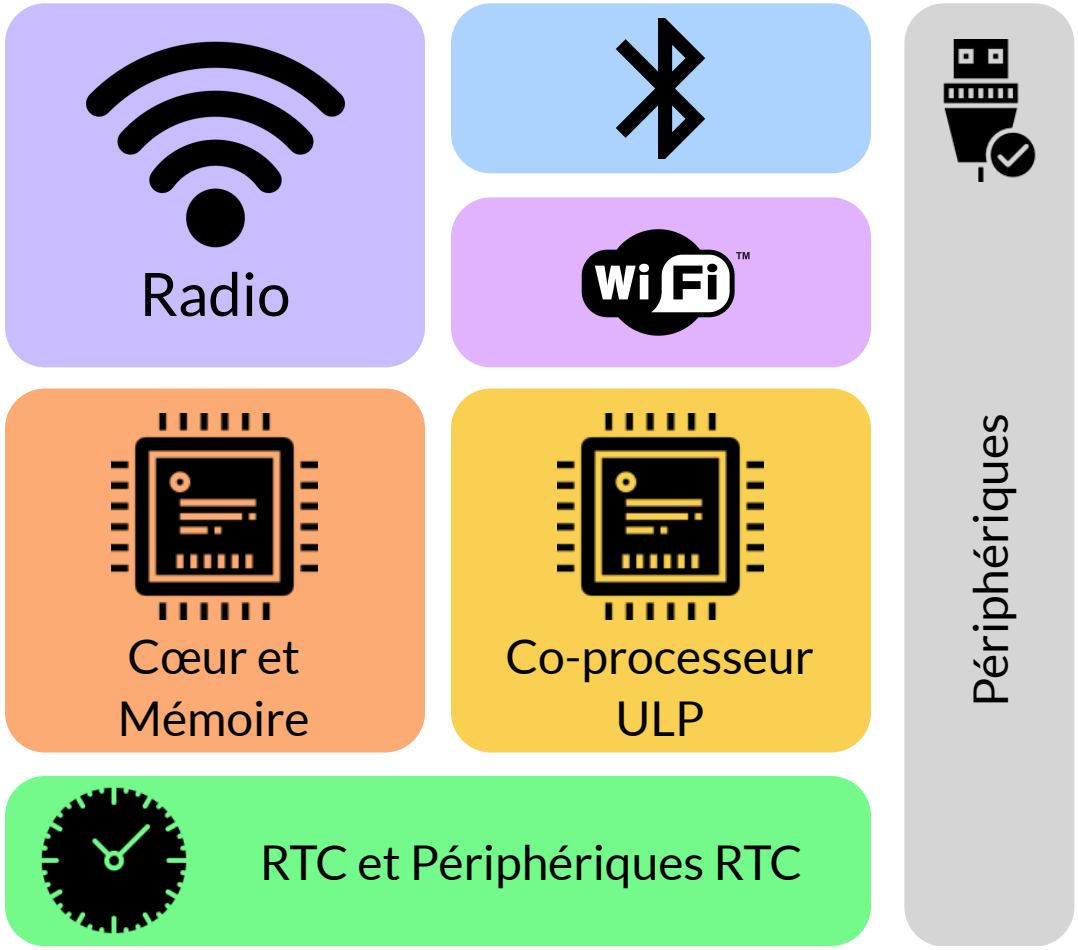
Cas du ESP32

Quelle architecture ?



Cas du ESP32

Simplifions les blocs



Icons made by srip from www.flaticon.com

D'après Wikipedia :

- Processor:
  - CPU: Xtensa dual-core 32-bit LX6 microprocessor
  - Ultra low power (ULP) co-processor
- Memory: 320 KiB RAM, 448 KiB ROM
- Wireless connectivity (shares the radio):
  - Wi-Fi: 802.11 b/g/n
  - Bluetooth: v4.2 BR/EDR and BLE
- Peripheral interfaces: GPIO, DAC/ADC, I2C, SPI, CAN, etc.
- Power management:
  - 5 µA deep sleep current
  - Wake up from GPIO interrupt, timer, ADC measurements, capacitive touch sensor interrupt

Pour optimiser la consommation
⇒ désactiver des blocs !

Sources

Last Minute ENGINEERS

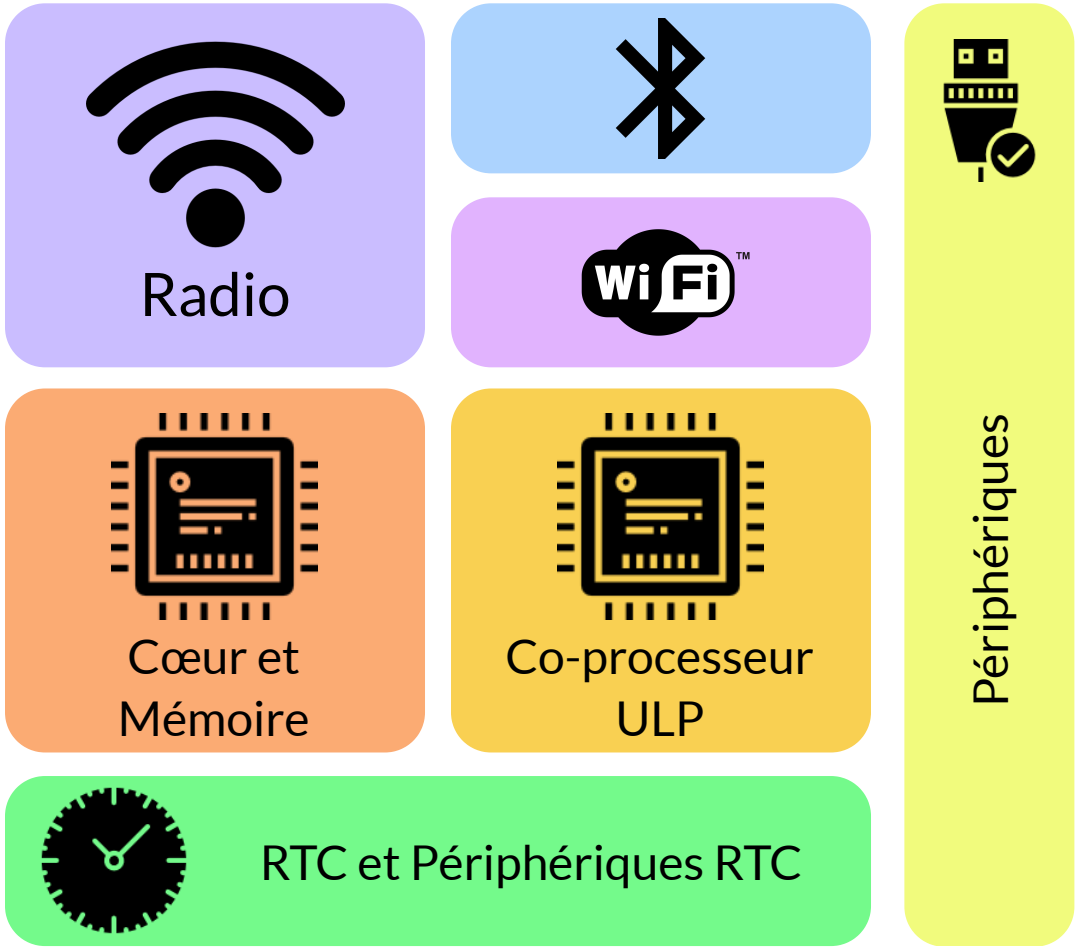
DiyIOt

instructables

ESP32 et modes de consommations

Mode "Actif"

Consommation : 160mA - 260mA



Icons made by srip from www.flaticon.com

Actif	Inactif
Tout est actif et accessible.	-

Mémoire

Fonctionnement normal.

Code

```
1  #define TIME_TO_SLEEP  2000
2  #define LED_PIN 2
3
4  void setup(){
5    pinMode(LED_PIN,OUTPUT);
6  }
7
8  void loop(){
9    digitalWrite(LED_PIN,HIGH);
10   delay(TIME_TO_SLEEP);
11   digitalWrite(LED_PIN,LOW);
12   delay(TIME_TO_SLEEP);
13 }
```

Sources

Last Minute ENGINEERS

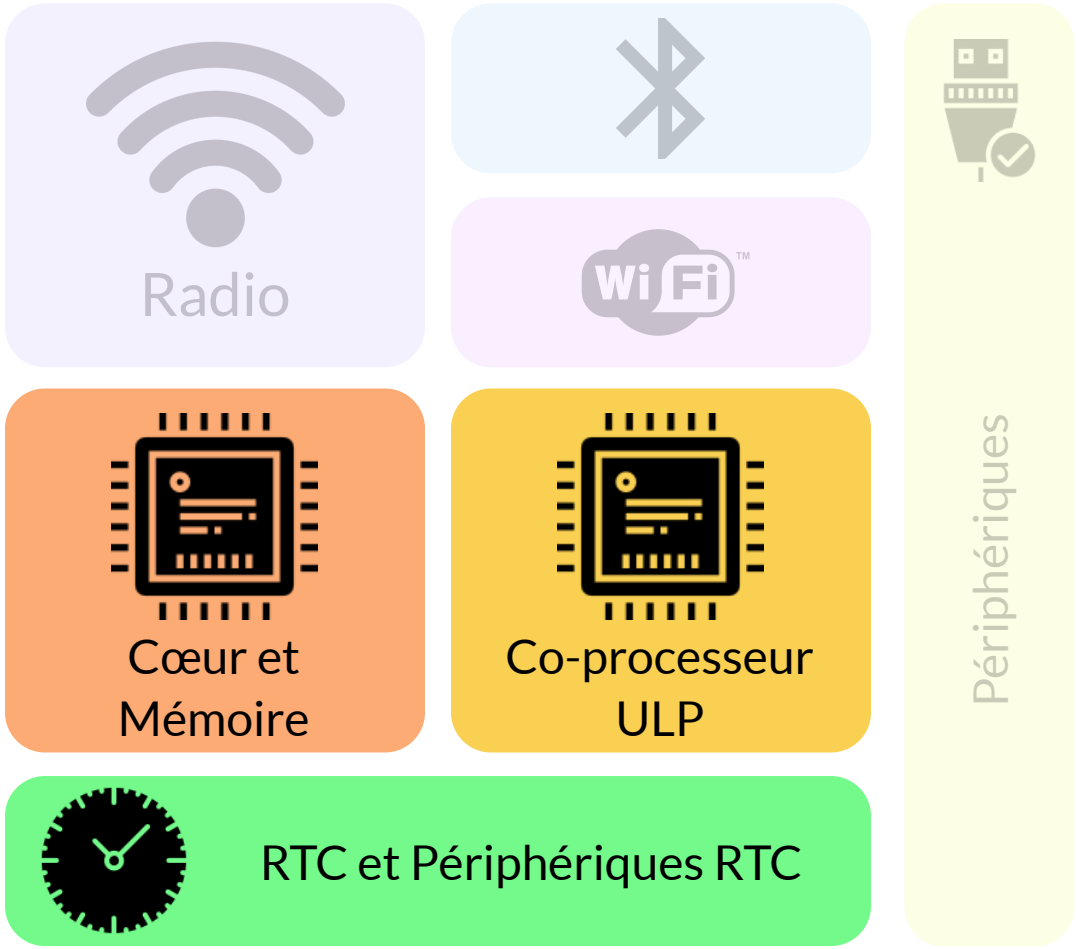
DiyIOt

instructables

ESP32 et modes de consommations

Mode "Light Sleep"

Consommation : 0.8mA



Icons made by srip from www.flaticon.com

Actif	Inactif
Cœur et Mémoire (Pause)	Radio
Co-processeur ULP	Bluetooth
RTC et Périphériques RTC	Wifi
	Périphériques

Mémoire

Fonctionnement "normal" : avant d'entrer en mode Light Sleep, l'ESP32 conserve son état interne et reprend son fonctionnement à la sortie du sommeil. On parle de **Full RAM Retention**.

Code

```
#define uS_TO_mS_FACTOR 1000ULL
#define TIME_TO_SLEEP  2000
#define LED_PIN 2

void setup(){
  pinMode(LED_PIN,OUTPUT);
  // ESP32 wakes after TIME_TO_SLEEP milliseconds
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_mS_FACTOR);
}

void loop(){
  digitalWrite(LED_PIN,HIGH);
  delay(TIME_TO_SLEEP);
  digitalWrite(LED_PIN,LOW);
  esp_light_sleep_start();
}
```

Sources

Last Minute ENGINEERS

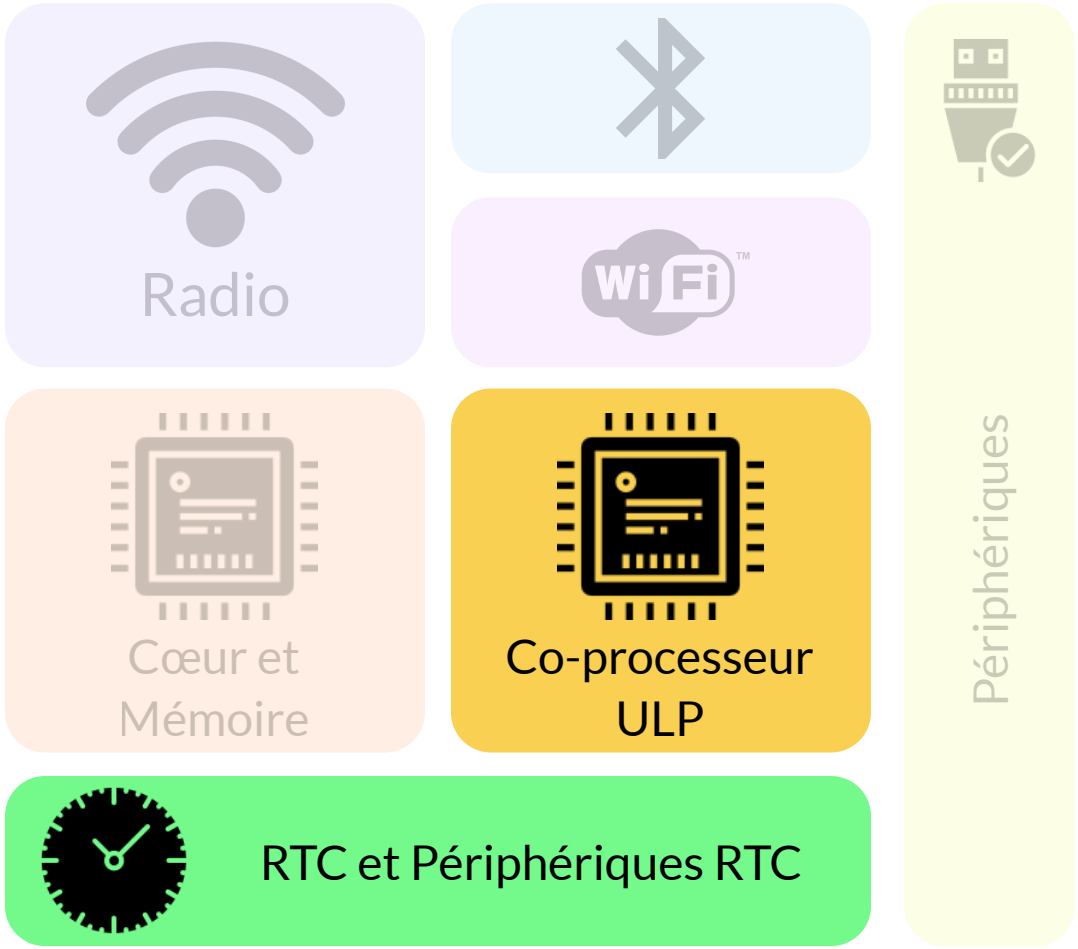
DiyIOt

instructables

ESP32 et modes de consommations

Mode "Deep Sleep"

Consommation : 10µA - 0.15mA



Icons made by srip from www.flaticon.com

Actif	Inactif
Co-processeur ULP	Cœur et Mémoire
RTC et Périphériques RTC	Radio
	Bluetooth
	Wifi
Mémoire	Périphériques

La mémoire est désactivée, donc tout ce qui y était stocké est effacé. Pour conserver des données, il faut déclarer la variable dans la "RTC Memory". Par exemple : ***RTC_DATA_ATTR int maVariable= 0;***

Code

```
1  // Après un Deep Sleep, le programme recommence au début
2  #define uS_TO_mS_FACTOR 1000ULL
3  #define TIME_TO_SLEEP  2000
4  #define LED_PIN 2
5
6  void setup(){
7    pinMode(LED_PIN,OUTPUT);
8    digitalWrite(LED_PIN,HIGH);
9    delay(TIME_TO_SLEEP);
10   digitalWrite(LED_PIN,LOW);
11   esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_mS_FACTOR);
12   esp_deep_sleep_start();
13 }
14
15 void loop(){
16 }
```

Sources

Last Minute ENGINEERS

DiyIOt

instructables

ESP32 et modes de consommations

Je dors, on me réveille... mais qui c'est ?!

Il est possible de savoir qui a causer le réveil du processeur avec la fonction "esp_sleep_get_wakeup_cause"
Elle renvoit la cause du réveil qui peut être de différents types :

- ESP_SLEEP_WAKEUP_EXT0, causé par un RTC_IO
- ESP_SLEEP_WAKEUP_EXT1, causé par un RTC_CNTL (par exemple se réveiller en fonction d'un masque de plusieurs broches).
- ESP_SLEEP_WAKEUP_TIMER, causé par un... timer.
- ESP_SLEEP_WAKEUP_TOUCHPAD, causé par un touchpad
- ESP_SLEEP_WAKEUP_ULP, causé par un programme exécuté par l'ULP.
- ESP_SLEEP_WAKEUP_GPIO, causé par un GPIO (seulement en light sleep)
- ESP_SLEEP_WAKEUP_UART, causé par l'UART (seulement en light sleep)

```
1  void setup(){
2    esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();
3
4    switch(wakeup_reason)
5    {
6      case ESP_SLEEP_WAKEUP_EXT0 : Serial.println("Wakeup caused by external signal using RTC_IO"); break;
7      case ESP_SLEEP_WAKEUP_EXT1 : Serial.println("Wakeup caused by external signal using RTC_CNTL"); break;
8      case ESP_SLEEP_WAKEUP_TIMER : Serial.println("Wakeup caused by timer"); break;
9      case ESP_SLEEP_WAKEUP_TOUCHPAD : Serial.println("Wakeup caused by touchpad"); break;
10     case ESP_SLEEP_WAKEUP_ULP : Serial.println("Wakeup caused by ULP program"); break;
11     default : Serial.printf("Wakeup was not caused by deep sleep: %d\n",wakeup_reason); break;
12   }
13 }
```

ESP32 et modes de consommations

Et comment je fais un code pour l'ULP, notamment à partir d'Arduino IDE ?

Il est possible de programmer l'ULP, mais il faut passer par des commandes spécifiques et de l'assembleur.
Une version plus "simple" est d'utiliser un système de macro (moins étendu que l'assembleur complet) pour créer un programme pour l'ULP.
À la suite, un exemple avec un programme ULP qui lit une entrée ADC et si elle est supérieure à un seuil, réveille le processeur principal. La valeur lue par l'ULP sur l'ADC est copiée dans une variable stockée dans la mémoire RTC.



Cette solution n'est plus disponible dans les dernières versions de espressif32.

Cela reste fonctionnel avec les versions suivantes :

- Arduino :
  2.0.12 de "`esp32 by Espressif`"
- PlatformIO :
  `platform=espressif32@6.4.0`

Des exemples plus complets avec ULP et Timers sont donnés dans les exemples de ce dépôt GitHub.

```
/*
   Based on :
   - https://docs.espressif.com/projects/esp-idf/en/v4.2.3/esp32/api-guides/ulp_macros.html
   - https://forum.arduino.cc/t/esp32-ulp-program-to-read-adc-when-in-deep-sleep/987884
   - https://github.com/SensorsIot/ESP32-ULP-Arduino-IDE
   - https://www.esp32.com/viewtopic.php?t=11215
*/
#include "esp32/ulp.h"
#include "driver/rtc_io.h"
#include "driver/adc.h"

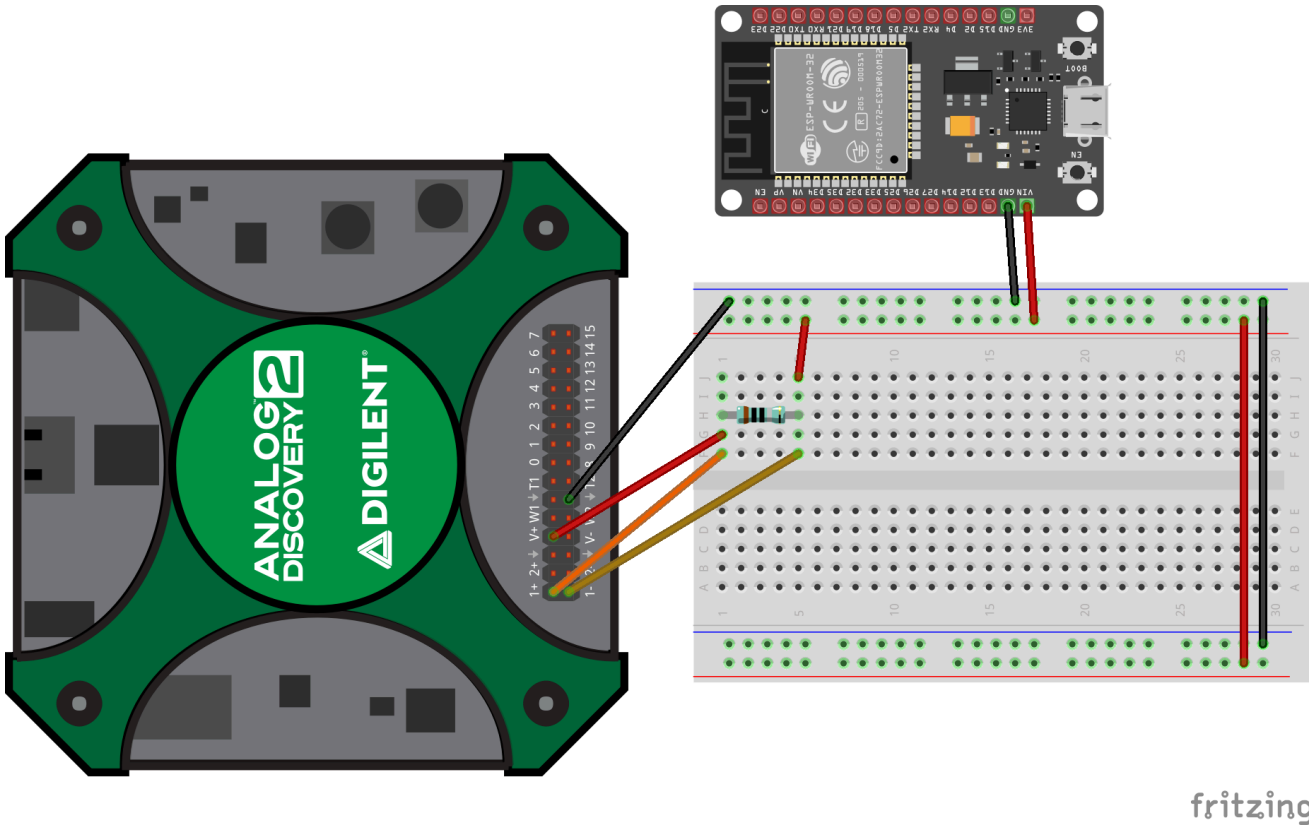
RTC_DATA_ATTR unsigned int ulp_adc_value = 0;
unsigned int ulp_adc_threshold = 1 * (4095 / 3.9);  // 1 volt

const ulp_insn_t ulp_adc_program[] = {
    I_DELAY(32000),                  // Wait until ESP32 goes to deep sleep
    M_LABEL(1),                      // LABEL 1
    I_ADC(R0, 0, 0),                 // Read ADC value to reg. R0
    M_BGE(2, ulp_adc_threshold),     // If average ADC value from reg. R0 is higher or equal than threshold, go to LABEL 2
    M_BX(1),                         // Go to LABEL 1
    M_LABEL(2),                      // LABEL 3
    I_MOVI(R1, ((unsigned int)&ulp_adc_value - (unsigned int)RTC_SLOW_MEM) / 4), // Set reg. R1 to adress of ulp_adc_value
    I_ST(R0, R1, 0),                                                             // Copy result of ADC to R1 adress, so ulp_adc_
    I_WAKE(),                                                                    // Wake up ESP32
    I_END(),                                                                     // Stop ULP program timer
    I_HALT()                                                                     // Halt the coprocessor
};

void setup()
```

Principes de mesure de consommation

Une résistance maîtrisée, pour une mesure précise



Matériel

- Diligent Analog Discovery 2 (Fritzing Part)
- Résistance de 1ohm (à mesurer précisément)
- Joy-It NodeMCU ESP32 (Fritzing Part)

Principe

- L'Analog Discovery 2 délivre du **5V** sur la broche **V+** (à configurer dans Diligent WaveForms.)
- La résistance va faire chuter la tension. Il faut s'assurer que dans le pire cas (consommation maximum de courant par l'ESP32), la tension reste suffisamment élevée pour l'ESP32.
- L'ESP32 va utiliser plus ou moins de courant suivant son mode de fonctionnement. La différence de potentiel aux bornes de la résistance va donc varier.
- Si **on mesure la tension aux bornes de la résistance** (Canal1 à configurer), et qu'on connait précisément la valeur de la résistance, on peut en déduire la consommation de l'ESP32
- Pour cela, on peut ajouter un "Math channel" sur Diligent WaveForms.



Image by Freepik

esiea
INGÉNIEUR·E·S D'UN NUMÉRIQUE UTILE

Plan du cours

On va faire quoi en fait ?

1. Introduction Générale

- Déroulé du cours
- (ré)Introduction à l'IoT

2. Objet : plateformes et architectures

- Plateformes et Architectures
- Cas du ESP32

3. Connectivité : réseaux et services Cloud

- Communication par bus et HAN/LAN/WAN
- Principe d'API
- Services Cloud

4. Objet : consommation

- ESP32 et modes de consommations
- Principes de mesure de consommation

5. PoC : à vous de jouer

PoC

De l'importance des spécifications… et de l'architecture



Imaginez un projet

Je veux :

- **mesurer et historiser l'humidité** d'un mur végétalisé et,
- **en fonction de** son humidité, la pluie attendue et la température prévue pour la journée
  ⇒ je souhaite **activer une pompe d'arrosage** pour que le taux d'**humidité ne descende pas en dessous d'un seuil** pendant la journée.

Spécifications

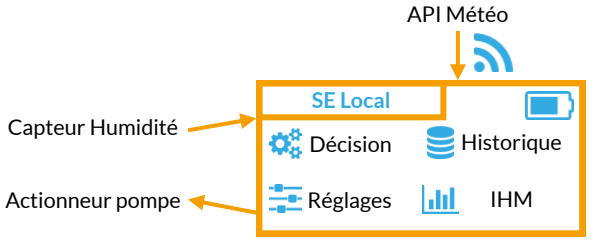
Quelles questions je dois me poser ?

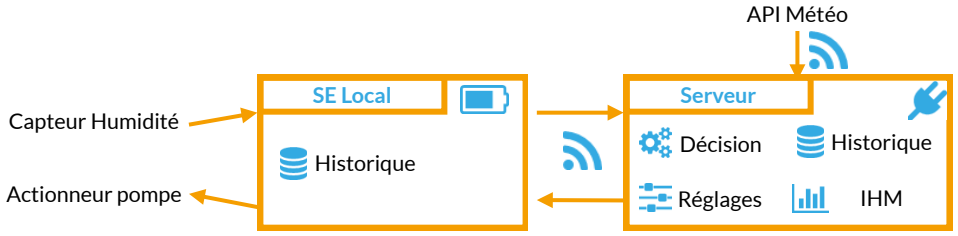
Exemples :

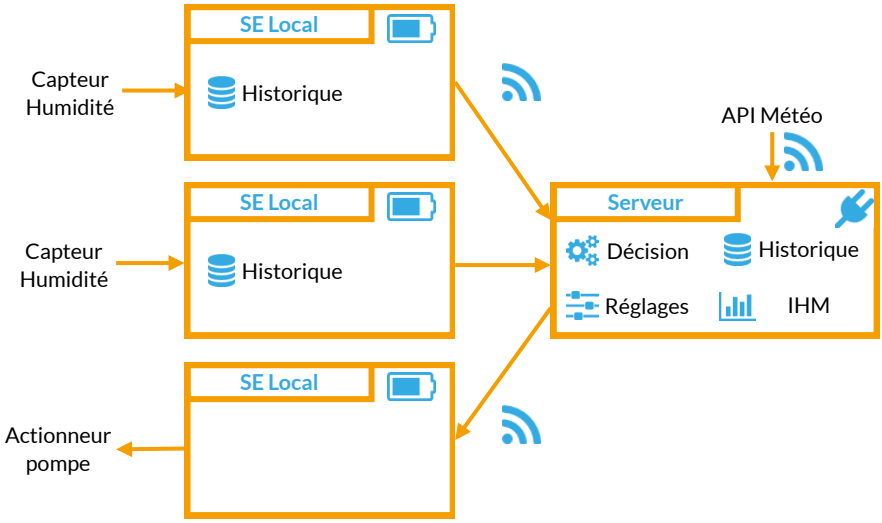
- Quelle précision pour les données ? Quelle fréquence de relevé ? Quelle délai de disponibilité ?
- Quelles sources d'énergie à proximité ? Quels accès à internet ?
- Comment régler les paramètres ? Comment voir les historiques ? Interface distante ?
- Combien de murs, quelles distances ?

Architectures

Quelles architectures possibles ?







PoC : Réfléchir en équipe

Définir un projet, choisir une architecture, réaliser un PoC



Man vector created by upklyak - www.freepik.com

Quelques contraintes

- 2 à 4 étudiants par équipe
- 1 fonctionnalité par étudiant (au moins)
- Le système doit être **connecté à son environnement direct** (capteurs, LEDs, moteur, écrans...)
- Le système doit être **connecté à l'extérieur** (API, serveur web, un autre objet, un email,...)
- L'architecture globale et le système doivent être pensés et codés pour optimiser la consommation énergétique

Rendus

- Rapport (10/15 pages)
- Présentation
- Dépôt Git **avec codes et tutoriels** pour reproduire l'exemple.

Timing

- Séance 4 - Choix de l'équipe et définition du projet. Validation par l'enseignant. Définition de l'architecture et avancement.
- Séance 5 - Finalisation du PoC.
- Séance 6 - Soutenances et évaluation par les paires.